

57. Denning, P. J.; Dennis, T.D.; and J. A. Brumfield, "Low Contention Semaphores and Ready Lists," *Communications of the ACM*, Vol. 24, No. 10, October 1981, pp. 687-699.
58. v Brinch Hansen, P., "Edison: A Multiprocessor Language," *Software Practice and Experience*, Vol. 11, No. 4, April 1981, pp. 325-361.
59. Linux kernel source code, version 2.5.75, <miner.cs.wm.edu/linux3/linux/http/source/lpc/sem.c?v=2.5.75>.
60. Hansen, P. Brinch, "Distributed Processes—a Concurrent Programming Concept," *Communications of the ACM*, Vol. 21, No. 11, November 1978, pp. 934-941.
61. Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Vol. 21, No. 7, July 1978, pp. 558-565.
62. Lamport, L., "The Implementation of Reliable Distributed Multiprocess Systems," *Computer Networks*, Vol. 2, No. 2, April 1978, pp. 95-114.
63. Ricart, G., and A. K. Agrawala, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, pp. 9-17.
64. Eisenberg, M. A., and M. R. McGuire, "Further Comments on Dijkstra's Concurrent Programming Control Problem," *Communications of the ACM*, Vol. 15, No. 11, November 1972, p. 999.
65. Andrews, G. R., and E B. Schneider, "Concepts and Notations for Concurrent Programming," *ACM Computing Surveys*, Vol. 15, No. 1, March 1983, pp. 3-44.
66. Bernstein, A. J., "Program Analysis for Parallel Processing," *IEEE Transactions on Computers*, Vol. 15, No. 5, October 1966, pp. 757-762.
67. Courtois, P. X; E Heymans; and D. L. Parnas, "Concurrent Control with Readers and Writers," *Communications of the ACM*, Vol. 14, No. 10, October 1971, pp. 667-668.
68. Lamport, L., "Concurrent Reading and Writing," *Communications of the ACM*, Vol. 20, No. 11, November 1977, pp. 806-811.
69. Ricart, G., and A. K. Agrawala, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, pp. 9-17.
70. Jones, D. W, "An Empirical Comparison of Priority Queue and Event Set Implementations," *Communications of the ACM*, Vol. 29, No. 4, April 1986, pp. 300-311.
71. Raynal, M., *Algorithms for Mutual Exclusion*, Cambridge, MA: MIT Press, 1986.
72. Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in E Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
73. Peterson, G. L., "Myths About the Mutual Exclusion Problem," *Information Processing Letters*, Vol. 12, No. 3, June 1981, pp. 115-116.
74. Lamport, L., "A New Solution of Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974, pp. 453-455.
75. Knuth, D., "Additional Comments on a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 321-322.
76. Eisenberg, M. A., and M. R. McGuire, "Further Comments on Dijkstra's Concurrent Programming Control Problem," *Communications of the ACM*, Vol. 15, No. 11, November 1972, p. 999.
77. Lamport, L., "A New Solution to Dijkstra's Concurrent Programming Problem," *Communications of the ACM*, Vol. 17, No. 8, August 1974, pp. 453-455.
78. Anderson, James H., and Yong-Jik Kim, "An Improved Lower Bound for the Time Complexity of Mutual Exclusion," *Proceedings of the Twentieth Annual ACM Symposium on Principles of Distributed Computing*, pp. 90-99, August 2001, Newport, Rhode Island, United States.
79. Dijkstra, E. W., "Cooperating Sequential Processes," Technological University, Eindhoven, Netherlands, 1965, reprinted in F Genuys, ed., *Programming Languages*, New York: Academic Press, 1968, pp. 43-112.
80. Denning, P. X; Dennis, T.D.; and J. A. Brumfield, "Low Contention Semaphores and Ready Lists," *Communications of the ACM*, Vol. 24, No. 10, October 1981, pp. 687-699.
81. Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *Communications of the ACM*, Vol. 21, No. 7, July 1978, pp. 558-565.
82. Lamport, L., "The Implementation of Reliable Distributed Multiprocess Systems," *Computer Networks*, Vol. 2, No. 2, April 1978, pp. 95-114.
83. Ricart, G, and A. K. Agrawala, "An Optimal Algorithm for Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 24, No. 1, January 1981, pp. 9-17.

High thoughts must have high language.

—Aristophanes—

As writers become more numerous, it is natural for readers to become more indolent.

—Oliver Goldsmith—

When the last reader reads no more.

—Oliver Wendell Holmes—

The first precept was never to accept a thing as true until I knew it as such without a single doubt.

—Rene Descartes—

This shows how much easier it is to be critical than to be correct.

—Benjamin Disraeli—

Chapter 6

Concurrent Programming

After reading this chapter, you should understand:

how monitors synchronize access to data.

how condition variables are used with monitors.

solutions for classic problems in concurrent programming such as readers and writers and circular buffer.

Java monitors.

remote procedure calls.

Chapter Outline

6.1

Introduction

Anecdote: Exhaustive Testing Is Impossible

6.2

Monitors

*Operating Systems Thinking:
Information Hiding*

6.2.1 Condition Variables

Biographical Note: *Per Brinch Hansen*

*6.2.2 Simple Resource Allocation with
Monitors*

6.2.3 Monitor Example: Circular Buffer

*6.2.4 Monitor Example: Readers and
Writers*

6.3

Java Monitors

6.4

Java Multithreading Case

Study: Part III:

*Producer/Consumer
Relationship in Java*

6.5

Java Multithreading Case

*Study, Part IV:
Circular Buffer in Java*

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited.

6.1 Introduction

In the last chapter we presented Dekker's Algorithm and Peterson's Algorithm for the implementation of mutual exclusion primitives, and we studied Dijkstra's semaphores. These mechanisms have a number of weaknesses. They are so primitive that it is difficult to use them to express solutions to more complex concurrency problems, and their presence in concurrent programs increases the already difficult problem of proving program correctness. Their malicious or accidental misuse could lead to subtle bugs that might corrupt the operation of a concurrent system.

The semaphore approach, in particular, has many weaknesses.¹ If a *P* operation is omitted, then mutual exclusion is not enforced. If a *V* operation is omitted, then threads waiting because of *P* operations could become deadlocked. Once the *P* operation begins, the thread cannot back out and take an alternate course of action while the semaphore remains in use. A thread may wait on only one semaphore at a time: this could lead to deadlock in resource-allocation situations. For example, two threads may each hold a resource that the other thread is waiting to use. This is the classic case of two-thread deadlock, which we will discuss in Chapter 7, Deadlock and Indefinite Postponement.

To combat this problem, researchers have developed higher-level mutual exclusion constructs that simplify solving complex concurrency problems, facilitate proving program correctness and are difficult for a programmer to misuse or corrupt.

Concurrent programming is much more difficult than sequential programming. Concurrent programs are harder to write, debug, modify, and prove correct (see the Anecdote, Exhaustive Testing Is Impossible). So why is the programming community so intent on concurrent programming?

There has been a surge of interest in concurrent programming languages because they enable us to express more natural solutions to problems that are inherently parallel. Furthermore, the true hardware parallelism possible with multiprocessors (see Chapter 17, Multiprocessor Management) and distributed systems (see Chapter 17, Introduction to Distributed Systems) can only be harnessed through concurrent programming. The potential applications for concurrent programming are numerous. There has been much discussion on concurrency in computer networks,² distributed systems^{3, 5, 6, 7} and real-time systems.^{8, 9, 10} Certainly, operating systems themselves are important examples of concurrent systems. So are air traffic control systems, mission-critical systems, and real-time process control systems (such as those that control gasoline refineries, chemical manufacturing plants and food processing plants). It is widely believed that human vision is an inherently parallel task. Weather forecasting will almost certainly take great strides forward when massive parallelism reaches the scale of billions or even trillions of concurrent processors.

In this chapter, we consider high-level constructs and languages for concurrent programming. In particular, we investigate monitors, condition variables, inter-process communication using remote procedure calls and the Java programming language's concurrent programming facilities. The pseudocode examples in

Section 6.2. Monitors, use a C-like syntax: the rest of the chapter uses complete Java programs. The chapter concludes with a lengthy literature section indicating the richness of concurrent programming as a research area.

What approach should implementors take when building concurrent systems today? Prior to the 1990s, high-level concurrent programming languages included to Ada.¹¹ Concurrent Pascal.^{12, 13} Distributed Processes,¹⁴ Concurrent C.¹⁵ Communicating Sequential Processes.^{16, 17} Modula-2.^{18, 19, 20} VAL²¹ and *MOD (for distributed programming).²² With the exception of Ada, these languages were generally



Anecdote

Exhaustive Testing Is Impossible

Testing is a hard problem. As you may study in courses on automata theory, a computer may be viewed as a finite state machine. How many states can a computer have? One possible view is that if the machine stores n bits, then the number of states is 2 raised to the n th power. A machine that has even a small main memory—perhaps tens of megabytes—could have far more states than there are atoms in the universe! Consider this experiment: A complex piece of software takes a 1 or 0 as input and yields a 1 or 0 as output. Each time the tester inputs a 1 for a large number of test cases, the software outputs a 1; each time the tester inputs a 0, the software outputs a 0. The

tester concludes that the software is simply passing its input through to its output and seems to be working fine. But suppose the software actually contains a counter of the number of times it has been called, and when the counter reaches a certain large value, it performs some other (possibly devastating) action. If in testing that software, it is called fewer than that number of times, then that test case will not occur and the software "problem" will not be detected.

Dijkstra emphasized that "testing reveals the presence of bugs, but not their absence." Since it is simply not possible to exhaustively test every system we build, this means that even after a system

has been "thoroughly" tested, it could still contain "lurking bugs" that may not surface until long after the system has been deployed. This places a huge burden on the shoulders of operating systems developers. When can we truly say that a system is ready for deployment? Obviously, this decision depends on the nature of the system. Is it a mission-critical or business-critical system? Are human lives at stake? Is it a special-purpose system whose use will be carefully confined to a narrow, controlled audience, or is it a general-purpose system that could be employed in a great diversity of applications, many of which cannot be determined when the system is under development.

Lesson to operating systems designers: Since the systems you build cannot possibly be tested exhaustively, you must "design for testability," minimizing the chance of bugs and providing testing capabilities to help system testers certify that your system is ready for deployment.

developed by academics for research purposes and tend to lack many of the features needed to implement real systems. Today, many popular programming languages support concurrency, including Java. C#. Visual C++ .NET. Visual Basic .NET and Python. Concurrent programming presents opportunities for computer science students as they prepare for careers in industry, where few people are experienced in this complex subject.

Self Review

1. Why have researchers sought higher-level mutual exclusion constructs?
2. What notable high-level concurrent programming language developed prior to the 1990s did indeed include the features needed to implement real systems?

Ans: 1) These constructs facilitate proving program correctness and are difficult to misuse or corrupt. 2) Ada.

6.2 Monitors

A **monitor** is an object that contains both the data and procedures needed to perform allocation of a particular **serially reusable** shared resource or group of serially reusable shared resources. The notion of a monitor was suggested by Dijkstra.²³ then by Brinch Hansen,^{24, 25} then refined by Hoare.²⁶ There has been much discussion in the literature on this important topic.^{27, 28, 29, 30, 31, 32, 33, 34, 35} Monitors have become an important software construct—in fact, the Java programming language makes extensive use of monitors to implement mutual exclusion.

To accomplish resource allocation using monitors, a thread must call a **monitor entry routine**. Many threads may want to enter the monitor at the same time, but mutual exclusion is rigidly enforced at the monitor boundary—only one thread at a time is allowed to enter. A thread that tries to enter the monitor when it is in use is made to wait by the monitor. Because the monitor guarantees mutual exclusion, concurrency problems (such as indeterminate outcomes) are avoided.

Data inside a monitor may be either global to all routines within the monitor or local to a specific routine. Monitor data is accessible only within the monitor: there is no way for threads outside the monitor to access monitor data. This is a form of **information hiding** a software architectural technique that improves modularity and facilitates the development of more reliable software systems (see the Operating Svstems Thinking feature. Information Hiding).

If a thread calls a monitor entry routine while no other threads are executing inside the monitor, the thread acquires a lock on the monitor and enters it. While the thread is in the monitor, other threads may not enter the monitor to acquire the resource. If a thread calls a monitor entry routine while the monitor is locked, the monitor makes the calling thread *wait* outside the monitor until the lock on the monitor is released (i.e., when a thread is no longer executing inside the monitor). Once inside the monitor, a thread might need to wait because the resource has been allocated to another thread. Because mutual exclusion is enforced at the boundary

250 Concurrent Programming

of the monitor, a thread waiting for a resource must do so outside the monitor to allow another thread into the monitor to return the resource.

Eventually, the thread that has the resource will call a monitor entry routine to release the resource. This routine could merely free the resource and wait for another requesting thread to arrive. But there may be threads waiting for the resource, so the monitor entry routine calls *signal* to allow one of the waiting threads to enter the monitor and acquire the resource. If a thread signals the return (also called the release) of the resource, and no threads are waiting, then the signal has no effect (but, of course, the monitor has recaptured the resource, which it can now allocate to arriving threads). To avoid indefinite postponement, a monitor gives a higher priority to waiting threads than to newly arriving ones.

Self Review

1. Why must a thread wait for a resource outside a monitor?
2. How does a monitor prevent multiple threads from concurrently executing inside the monitor?

Ans: 1) If the thread were to wait inside a monitor for a resource, no other threads could enter the monitor to return the resource. As we will see in Chapter 7. Deadlock and Indefinite Postponement, this could cause all the threads waiting for the resource to deadlock. 2) The monitor enforces mutual exclusion at its border using the kinds of techniques discussed in Chapter 5. Asynchronous Concurrent Execution.



Operating Systems Thinking

Information Hiding

Information hiding is one of the most fundamental techniques of software engineering. It is implemented in many ways in operating systems. When one function or method calls another, the caller does not need to know the details of how the callee is implemented; the caller merely has to know the interface to the callee—what arguments must be passed and in what order, and what return val-

ues are expected. Information hiding has many advantages. It makes the caller's job easier. It does not need to be familiar with the (possibly enormous) complexity of how the callee is implemented. It also makes it easier to modify systems—a called function can usually be replaced easily without requiring changes in the caller, as long as the interface to the callee remains the same.

Today's operating systems arguably have tens of thousands (or more) components that are constantly being evolved to adapt to new hardware and software trends and tuned to perform better. Information hiding places a crucial role in making these enormous and highly modular systems understandable and maintainable.

6.2.1 Condition Variables

Monitors implement both mutual exclusion and synchronization between threads of execution. A thread currently inside a monitor may need to wait outside the monitor until another thread performs an action inside the monitor. For example, in the producer/consumer relationship, the producer finding that the consumer still has not read the value in a single shared buffer must wait outside the monitor governing the shared buffer so that the consumer can consume the buffer contents. Similarly, a consumer finding that the shared buffer is empty must wait outside the monitor until the producer fills the buffer. A thread inside a monitor uses a **condition**

variable to wait on a condition outside the monitor. A monitor associates a separate condition variable with each distinct situation that might cause a thread to have to wait. We define the wait and signal operations as:

```
wait (condition Variable)
signal (conditionVariable)
```

Condition variables are different from "conventional" variables. Every condition variable has an associated queue. A thread calling **wait** on a particular condition variable is placed into the queue associated with that condition variable (while in that queue, the thread is outside the monitor, so that another thread can eventually enter the monitor and call **signal**). A thread calling **signal** on a particular condition variable causes a thread waiting on that condition variable to be removed from the queue associated with it and to reenter the monitor. We may assume a first-in-first-out (FIFO) queue discipline, although priority schemes can be useful in certain situations.^{36,37}

Before that thread can reenter the monitor, the thread calling **signal** must first exit the monitor. Brinch Hansen (see the Biographical Note. Per Brinch Hansen), noting that many **signal** statements immediately preceded a return statement (i.e., the thread exits the monitor), proposed a **signal-and-exit** monitor, in which a thread immediately exits the monitor upon signaling.³⁸ The monitors in the following examples are signal-and-exit monitors. Alternatively, a **signal-and-continue**

monitor allows a thread inside the monitor to signal that the monitor will soon become available, but still maintain a lock on the monitor until the thread exits the monitor. A thread can exit the monitor by waiting on a condition variable or by completing execution of the code protected by the monitor. The thread released by a signal-and-continue monitor must wait until the signalling thread exits the monitor. As we discuss in Section 6.3. Java Monitors, the Java programming language implements signal-and-continue monitors.

Self Review

1. What problem might occur if a priority queue (instead of a FIFO queue) were used for a condition variable?
2. (T/F) Each monitor contains exactly one condition variable.

252 Concurrent Programming

Ans: 1) A lower-priority thread could be indefinitely postponed by a stream of higher-priority threads calling wait in the monitor enter the priority queue. 2) False. A monitor contains a separate condition variable for each distinct situation that might cause a thread to call wait in the monitor.

6.2.2 Simple Resource Allocation with Monitors

Suppose several threads are contending for a resource that requires exclusive access. A simple monitor for handling the allocation and deallocation of such a resource is shown in Fig. 6.1.

Line 4 declares state variable `inUse` which keeps track of whether the resource is in use or not. Line 5 declares a condition variable that is waited on by a thread finding the resource unavailable and signaled by a thread returning the resource (and thus making it available). Lines 7-17 and 19-25 declare two monitor entry routines. To indicate that these are monitor entry routines (rather than routines private to the monitor), we prefix each routine with the pseudocode keyword `monitorEntry`.



Biographical Note

Per Brinch Hansen

Per Brinch Hansen has devoted his research to concurrent programming, which is the technique of designing software that has parallel paths of execution. He received his Masters in Electrical Engineering from the Technical University of Denmark in 1962 and has been a professor at the California Institute of Technology, the University of Southern California and the University of Copenhagen, Denmark. He is currently a professor of Electrical Engineering and Computer Science at Syracuse University.³⁹

While working at the Regne-centralen, Copenhagen, Brinch

Hansen developed the RC 4000 Multiprogramming System for the RC (RegneCentralen) 4000 computer. This operating system was the first to separate system functions into a kernel module.⁴⁰

Brinch Hansen introduced the concept of the monitor construct—an object that encapsulates both shared data and the procedures that operate on that shared data—in his book, *Operating Systems Principles*, published by Prentice Hall in 1973 (see Section 6.2, Monitors).⁴¹ He incorporated monitors into his Concurrent Pascal language (an extension of Pascal, which

allowed only sequential programming), for users to define access controls on shared data so that programs written in Concurrent Pascal could be checked at compile time for sharing violations.⁴²

As a demonstration of Concurrent Pascal, Brinch Hansen used it to write **Solo**, a simple operating system that was secure (meaning that errors would be detected by the system instead of crashing the program—not the more common meaning of safe from attacks) without using hardware support.^{43, 44, 45} He later designed the concurrent programming language Joyce.⁴⁶

```

1 // Fig. 6.1: Resource allocator monitor
2
3 // monitor initialization (performed only once)
4 boolean inUse = false; // simple state variable
5 Condition available; // condition variable
6
7 // request resource
8 monitorEntry void getResource()
9 {
10     if ( inUse ) // is resource in use?
11     {
12         wait(available); // wait until available is signalled
13 } // end if
14
15 inUse = true; // indicates resource is now in use
16
17 } // end getResource
18
19 // return resource
20 monitorEntry void returnResource()
21 {
22 inUse = false; // indicate resource is not in use
23 signal( available ); // signal a waiting thread to process
24
25 } // end returnResource

```

Figure 6.1 | Simple resource allocation with a monitor in pseudocode.

Line 8 begins method `getResource`, which a thread calls to request the resource associated with the condition variable `available`. Line 10 tests the state variable `inUse` to see whether the resource is in use. If this value is true, meaning that the resource has been allocated to another thread, the calling thread must wait on condition variable `available` (line 12). After calling `wait`, the thread exits the monitor and is placed in the queue associated with the condition variable `available`. As we will see, this allows the thread that is using the resource to enter the monitor and release the resource by signaling condition variable `available`. When the resource is not in use, the requesting thread executes line 15, which sets `inUse` to true, providing the thread exclusive access to the resource.

Line 20 begins method `returnResource`, which a thread calls to release the resource. Line 22 sets the value of `inUse` to false to indicate that the resource is no longer in use and can be allocated to another thread. Line 23 calls `signal` on condition variable `available` to alert any waiting thread that the resource is now free. If there are any threads waiting in the queue associated with `available` (as a result of executing line 12), the next waiting thread reenters the monitor and executes line 15, obtaining exclusive access to the resource. If there are no threads waiting on `available`, `signal` has no effect.

The beauty of the monitor in Fig. 6.1 is that it performs exactly as a binary semaphore: method `getResource` acts as the *P* operation; method `returnResource` acts as the *V* operation. Because the simple one-resource monitor can be used to implement semaphores, monitors are at least as powerful as semaphores. Note that the monitor initialization (lines 3-5) is performed before threads begin using the monitor; in this case `inUse` is set to `false` to indicate that the resource is initially available.

Self Review

1. What would happen if `returnResource()` did not signal the condition variable `avalable`?
2. What purpose does the keyword `monitorEntry` serve?

Ans: 1) All of the threads waiting on condition variable `avalable` would deadlock—they will wait forever. 2) Keyword `monitorEntry` distinguishes between monitor entry routines, which are accessible by all threads and can be executed by only one thread at a time, and private monitor methods, which are accessible only within the monitor.

6.2.3 Monitor Example: Circular Buffer

In this section we discuss the **circular buffer** (sometimes called the **bounded buffer**.) and how it is useful in situations in which a producer thread passes data to a consumer thread. Because the producer and consumer access the same data and may operate at different rates, synchronization between the two threads is essential.

We can use an array to implement the circular buffer. In the circular buffer implementation of the solution to the producer/consumer problem, the producer deposits data in the successive elements of the array. The consumer removes them in the order in which they were deposited (FIFO). The producer can be several items ahead of the consumer. Eventually, the producer fills the last element of the array. When it produces more data, it must "wrap around" and again begin depositing data in the first element of the array (assuming, of course, that the consumer has removed the data previously put there by the producer). The array effectively closes in a circle, hence the term circular buffer.

Because of the fixed size of the circular buffer, the producer will occasionally find all the array elements full: in this case the producer must wait until the consumer empties an array element. Similarly, there will be times when the consumer wants to consume, but the array will be empty: in this case the consumer must wait until the producer deposits data into an array element. The monitor in Fig. 6.2 (based on the example presented by Hoare)⁴⁷ implements a circular buffer and the appropriate synchronization mechanisms to handle the producer/consumer relationship.

```
1 // Fig. 6.2: Circular buffer monitor
2
```

Figure 6.2 | Monitor pseudocode implementation of a circular buffer. (Part 1 of 2.)

```

3 char circularBuffer[] = new char[ BUFFER_SIZE ]; // buffer
4 int writerPosition = 0; // next slot to write to
5 int readerPosition = 0; // next slot to read from
6 int occupiedSlots = 0; // number of slots with data
7 Condition hasData; // condition variable
8 Condition hasSpace; // condition variable
9
10 // monitor entry called by producer to write data
11 monitorEntry void putChar( char slotData )
12 {
13     // wait on condition variable hasSpace if buffer is full
14     if ( occupiedSlots == BUFFER_SIZE )
15     {
16         wait( hasSpace ); // wait until hasSpace is signaled
17     } // end if
18
19     // write character to buffer
20     circularBuffer[ writerPosition ] = slotData;
21     ++occupiedSlots; // one more slot has data
22     writerPosition = (writerPosition + 1) % BUFFER_SIZE;
23     signal( hasData ); // signal that data is available
24 } // end putChar
25
26 // monitor entry called by consumer to read data
27 monitorEntry void getChar( outputParameter slotData )
28 {
29     // wait on condition variable hasData if the buffer is empty
30     if ( occupiedSlots == 0 )
31     {
32         wait( hasData ); // wait until hasData is signaled
33     } // end if
34
35     // read character from buffer into output parameter slotData
36     slotData = circularBuffer[ readerPosition ];
37     occupiedSlots--; // one fewer slots has data
38     readerPosition = (readerPosition + 1) % BUFFER_SIZE;
39     signal( hasSpace ); // signal that character has been read
40 } // end getChar

```

Figure 6.2 | Monitor pseudocode implementation of a circular buffer. (Part 2 of 2.)

We shall assume that the array `circularBuffer` contains `BUFFER_SIZE` entries consisting of one character (line 3). Variables `writerPosition` and `readerPosition` (lines 4-5) indicate in which slot of the circular buffer the next item is to be placed by a producer and from which slot of the circular buffer the next item is to be removed by a consumer, respectively.

A producer adds data to the buffer by calling method `putChar` (lines 10-24). Line 14 tests if the buffer is full. If it is, the producer waits on the condition variable `hasSpace` (line 16). When it does this, the producer leaves the monitor and waits in the queue associated with the condition variable `hasSpace`. As we will soon see, this allows a consumer to enter the monitor, consume data in the circular buffer and signal condition variable `hasSpace`. This enables the producer to continue; it writes the data to the buffer (line 20), increments the number of occupied slots (line 21) and updates the `writerPosition` variable (line 22). Finally, line 23 signals condition variable `hasData` to enable a waiting consumer, if there is one, to proceed.

A consumer reads data from the buffer by calling method `getChar` (lines 26-40). Note the use of the output parameter `slotData` in the pseudocode (indicated by keyword `outputParameter` in line 27). Many languages support such a capability. Data written to the output parameter is immediately available in the caller's argument. Line 30 tests if the buffer is empty. If it is, the consumer waits on condition variable `hasData` (line 32). When the consumer can continue, it reads the data from the buffer directly into the output parameter `slotData` (line 36), immediately making the data available in the caller's argument. The consumer then decrements the number of occupied slots (line 37) and updates `readerPosition` (line 38). Line 39 signals condition variable `hasSpace` to enable a waiting producer, if there is one, to proceed.

The beauty of the circular buffer is that it allows the producer to "get ahead" of the consumer. The producer can create a new value without waiting for the consumer to read the previous value (as is necessary in the single buffer producer/consumer relationship). These extra values are placed in the empty slots of the circular buffer. The consumer will still read the values in the correct order. The circular buffer reduces the amount of time the producer must wait before producing another value, thus improving system performance. The larger the circular buffer, the more values the producer can produce before it must wait for the consumer to empty the buffer. If the producer and consumer work at approximately the same speed, using the circular buffer can increase the average speed of the application. If there is a difference in the average speeds of the threads, this advantage is nullified. For example, if the producer works consistently faster than the consumer, the circular buffer will quickly become full and stay full, forcing the producer to wait each time for the consumer to free up space. Similarly, if the consumer works consistently faster than the producer, then the consumer will typically find the circular buffer empty and will almost always have to wait for the producer to create a value. In these last two cases, using the circular buffer will simply waste memory rather than increasing the speed of the application.

Operating systems can use a circular buffer to implement spooling control. One common example of spooling occurs when a thread generates lines to be printed on a relatively slow output device such as a printer. Because the thread can produce the lines much faster than the printer can print them, and because we would like the thread to be able to complete its execution as quickly as possible, the thread's output lines can be directed to a circular buffer. The circular buffer may be

in primary storage, or more likely on disk. The thread creating the lines to be printed is often called a spooler. Another thread reads the lines from the `circularBuffer` and writes them to the printer. But this second thread, often called a despooler, runs at the slower, speed of the printer. The `circularBuffer` has sufficient storage to "take up the slack" resulting from the mismatch in the speeds of the spooler and despooler threads. Of course, we assume the system does not indefinitely generate print lines faster than the printer can print them; if it did, the buffer would always be full and would be of little value in "smoothing" the printing operation.

Self Review

1. Name the two situations in which a circular buffer is less efficient than a single-value buffer in the producer/consumer relationship.
2. Would this monitor support one producer and two consumers such that each element that the producer produces is consumed by exactly one consumer? Why or why not?

Ans: 1) If the difference in the average speeds of the producer and consumer is significant, the advantage of the circular buffer is nullified. If the producer consistently ran faster than the consumer, it would almost always find the buffer full and would have to wait each time for a consumer to consume. If the consumer consistently ran faster than the producer, it would almost always find the buffer empty and would have to wait each time for a producer to produce. In these cases, a single-value buffer can be used without loss of performance. **2)** Yes. Mutual exclusion would still be enforced by the monitor. The producer would still produce values as normal. Only one consumer would read each piece of data from the producer.

6.2.4 Monitor Example: Readers and Writers

In computer systems, it is common to have some consumer threads (called readers) that read data and producer threads (called writers) that write it. For example, in an airline reservation system there may be many more readers than writers—many inquiries will be made against the database of available flight information before the customer actually selects and commits to a particular seat on a particular flight.

Because readers do not change the contents of the database, many readers may access the database at once. But a writer can modify the data, so it must have exclusive access. When a writer is active, no other readers or writers may be active. This exclusion needs to be enforced only at the record level. It is not necessary to grant a writer exclusive access to the entire database—doing so could dramatically hurt performance.

The problem of designing a concurrent program to control access of readers and writers to a database was first posed and solved by Courtois, Heymans, and Parnas.⁴⁸ The solution in Fig. 6.3 is based on that developed by Hoare.⁴⁹

The monitor in Figure 6.3 may be used to control access to an entire database, a subset of the database consisting of many or few records, or even a single record. In any of these cases, the following discussion applies. Only one writer may be active at a time; when a writer is active, the boolean variable `wri teLock` (line 4) is true. No readers may be active when a writer is active. The integer variable, `readers` (line 3), indicates the number of active readers. When the number of readers is

```

1 // Fig. 6.3: Readers/writers problem
2
3 int readers = 0; // number of readers
4 boolean writeLock = false; // true if a writer is writing
5 Condition canWrite; // condition variable
6 Condition canRead; // condition variable
7
8 // monitor entry called before performing read
9 monitorEntry void beginRead()
10 {
11     // wait outside monitor if writer is currently writing or if
12     // writers are currently waiting to write
13     if ( writeLock || queue( canWrite ) )
14     {
15         wait( canRead ); // wait until reading is allowed
16     } // end if
17
18     ++readers; // there is another reader
19
20     signal( canRead ); // allow waiting readers to proceed
21 } // end beginRead
22
23 // monitor entry called after reading
24 monitorEntry void endRead()
25 {
26     --readers; // there are one fewer readers
27
28     //if no more readers are reading, allow a writer to write
29     if ( readers == 0 )
30     {
31         signal ( canWrite ); // allow a writer to proceed
32     } // end if
33
34 } // end endRead
35
36 // monitor entry called before performing write
37 monitorEntry void beginWrite()
38 {
39     // wait if readers are reading or if a writer is writing
40     if ( readers > 0 || writeLock )
41     {
42         wait( canWrite ); // wait until writing is allowed
43     } // end if
44

```

Figure 6.3 | Monitor pseudocode for solving the readers and writers problem.
(Part 1 of 2.)


```

45     writeLock = true; // "lock out all readers and writers
46 } // end beginWrite
47
48 // monitor entry called after performing write
49 monitorEntry void endWrite()
50 {
51     writeLock = false; // release lock
52
53     // if a reader is waiting to enter, signal a reader
54     if ( queue( canRead ) )
55     {
56         signal( canRead ); // cascade in waiting readers
57     } // end if
58     else // signal a writer if no readers are waiting
59     {
60         signal( canWrite ); // one waiting writer can proceed
61     } // end else
62
63 } // end endWrite

```

Figure 6.3 | Monitor pseudocode for solving the readers and Writers problem.
(Part 2 of 2.)

reduced to zero (line 29), then one waiting writer (if there is one) becomes active (line 31). If a new reader cannot proceed (line 13), it waits on the condition variable `canRead` (line 15). If a new writer cannot proceed (line 40), it waits on the condition variable `canWrite` (line 42).

When a reader wishes to read, it calls monitor entry `beginRead` (lines 8-21); a reader that has finished calls `endRead` (lines 23-34). In `beginRead`, a new reader can proceed as long as no thread is writing and no writer thread is waiting to write (line 13). The latter condition is important for preventing indefinite postponement of waiting writers; it is tested by using the boolean function, `queue`, which determines whether or not threads are waiting on the condition variable specified in its argument. Note that procedure `beginRead` ends by signaling `canRead` (line 20) to allow another waiting reader to begin reading. This causes the next reader in the queue of waiting readers to become active and, in turn, signal the next waiting reader to proceed. This "chain reaction," also called a cascade, will continue until all waiting readers have become active. While this chaining is progressing, all arriving threads are forced to wait because the monitor observes the rule that signaled threads are serviced before arriving threads. If arriving readers were allowed to proceed, then a continuous stream of arriving readers would cause indefinite postponement of waiting writers. Since the readers do not interfere with one another and can be run in parallel on multiprocessor systems, this is an efficient way to service these threads.

When a thread is done reading, it calls monitor entry `endRead` (lines 23-34), which decrements the number of readers by 1 (line 26). Eventually, this decrementing causes the number of readers to become zero, at which point the thread signals

canWrite (line 31) to allow a waiting writer, if there is one, to proceed (preventing indefinite postponement of waiting writers).

When a thread wishes to write, it calls monitor entry beginWrite (lines 36-46). Because a writer must have exclusive access, if there are any readers or if there is an active writer (line 40), the new writer must wait on condition variable canWrite (line 42). When the writer is able to proceed (because canWrite is signaled in line 31 or line 60), writeLock is set to true (line 45). This keeps out any other readers and writers.

When a writer finishes, it calls monitor entry endWrite (lines 48-63). This procedure sets writeLock to false (line 51) so that either readers or another writer can become active. The monitor then must signal another waiting thread to proceed (lines 53-61). Should it give preference to a waiting reader or a waiting writer? If it gives preference to a waiting writer, then it will be possible for a steady stream of incoming writers to cause the indefinite postponement of waiting readers. Therefore, as a writer finishes, it first checks if there is a waiting reader (line 54). If so, then canRead is signaled (line 56) and the waiting reader proceeds (and this, of course, cascades in all waiting readers). If there is no waiting reader, then canWrite is signaled (line 60) and a waiting writer is allowed to proceed.

SelfReview

1. Figure 6.3 cascades in waiting readers, but never cascades in waiting writers. Why?
2. What safeguard is there against a steady stream of readers indefinitely postponing a writer?

Ans: **1)** Readers don't interfere with one another, so many readers can safely read at once. Each writer, however, must have exclusive access, or indeterminate results could occur. **2)** In monitor entry beginRead, if there are any writers waiting to write (line 13), then a thread that wishes to read must wait on condition variable canRead (line 15).

6.3 Java Monitors

In the following sections, we present complete, working Java multithreaded solutions to common concurrent programming problems. Monitors are associated with every object created in Java. Monitors are also the primary mechanism to provide mutual exclusion and synchronization in multithreaded Java applications. The keyword **synchronized** imposes mutual exclusion on an object in Java. In this section, we explain how Java monitors differ from the pseudocode monitors discussed in the previous section.

When a thread attempts to execute a method protected by a Java monitor (i.e., the method is declared **synchronized**), it must first enter that monitor's **entry set** (commonly referred to as an **entry queue**), which is a queue of threads awaiting entry into the monitor. If there is no contention for entry into the monitor, a thread will immediately enter the monitor. If a thread is already inside the monitor, other threads must remain in the entry set until the monitor becomes available.

Java monitors are typically referred to as signal-and-continue monitors.⁵⁰ Recall that a signal-and-continue monitor allows a thread inside the monitor to signal that the monitor will soon become available, but still maintain a lock on the monitor until it exits. A thread can exit the monitor by waiting on a condition variable or by completing execution of the code protected by the monitor.

A thread, executing in a monitor, that must wait on a condition variable issues the wait call. The wait method causes the thread to release the lock on the monitor and wait on an unnamed condition variable. After calling wait, a thread is placed in the **wait set** (also commonly referred to as the wait queue), a queue of threads waiting to reenter the monitor for the object. Threads remain in the wait set until signaled (notified) by another thread. Because the condition variable is implicit in Java, a thread may be signaled, reenter the monitor and find that the condition on which it waited has not been met. Consequently, a thread may be signaled several times before the condition on which it is waiting is met.

Threads issue signals by calling method notify or notifyAll. The notify method wakes a single thread in the wait set. The algorithm that determines which thread will enter the monitor next varies, depending on the Java virtual machine (JVM) implementation. As a result, the programmer cannot rely on a particular queuing discipline when notify is called.

Another pitfall of the notify method is that the order in which some JVM implementations remove threads from the entry and wait sets might introduce lengthy service delays for particular threads in these sets—creating the possibility of indefinite postponement. As a result, if more than two threads access a monitor, it is best to use the **notifyAll** method, which wakes all threads in the entry and wait sets. When all threads are awake, the thread scheduler determines which thread acquires the monitor. The thread scheduler employs a queue discipline that prevents indefinite postponement.^{51, 52} Because notifyAll wakes all threads waiting to enter the monitor (as opposed to notify, which wakes a single thread), notifyAll incurs more overhead than notify. In the simple case of two-thread synchronization, notify yields higher performance than notifyAll without suffering from indefinite postponement.

Self Review

1. What is the difference between signal-and-continue monitors and signal-and-exit monitors? Which type does Java use?

Ans: 1) Java uses signal-and-continue monitors. They allow a thread to signal that it will soon be exiting the monitor, yet retain control of the monitor. Signal-and-exit monitors require that a thread release its lock on the monitor immediately after signaling.

6.4 Java Multithreading Case Study, Part III: Producer/Consumer Relationship in Java

In this section, we present a Java implementation of the producer/consumer relationship investigated in Section 5.2.1, Java Multithreading Case Study, Part II. The

application in Fig. 6.4 and Fig. 6.5 demonstrates a producer and a consumer accessing a single shared buffer, with synchronization provided by a monitor. In this case, the producer produces a new value only when the buffer is empty, making the buffer full; the consumer consumes a value only when the buffer is full, making the buffer empty. This example reuses interface `Buffer` (Fig. 5.1) and classes `Producer` (Fig. 5.2) and `Consumer` (Fig. 5.3) from the example of Section 5.2.1. Reusing these classes from the example without synchronization enables us to demonstrate that the threads accessing the shared object are unaware that they are being synchronized. The code that performs the synchronization is placed in the set and get methods of class `SynchronizedBuffer` (Fig. 6.4), which implements interface `Buffer` (line 4). Thus, the `Producer`'s and `Consumer`'s run methods simply call the shared object's set and get methods as in the example of Section 5.2.1.

Class `SynchronizedBuffer` (Fig. 6.4) contains two fields—`buffer` (line 6) and `occupiedBuffers` (line 7). Method `set` (lines 9-43) and method `get` (lines 45-79) are now synchronized methods (lines 10 and 46); thus, only one thread can enter any of these methods at a time on a particular `SynchronizedBuffer` object. Although `occupiedBuffers` (line 7) is logically a condition variable, it is of type `int`; there is no object or type in Java that directly represents a condition variable. The methods in `SynchronizedBuffer` use `occupiedBuffers` in conditional expressions (lines 16 and 52) to determine whether it is the producer's or the consumer's turn to perform a task. If `occupiedBuffers` is zero, buffer is empty and the producer can place a value into buffer (line 34). This condition (line 52) also means that the consumer must wait (line 59) in the `get` method to read the value of buffer (again, because it is empty). If `occupiedBuffers` is one, the consumer can read the value from buffer, because it contains new information. This condition (line 16) also means that the producer must wait to place a value into buffer, because it is currently full.

```

1  // Fig. 6.4: SynchronizedBuffer.java
2  // SynchronizedBuffer synchronizes access to a shared integer.
3
4  public class SynchronizedBuffer implements Buffer
5  {
6      private int buffer = -1; // shared by producer and consumer
7      private int occupiedBuffers = 0; // counts occupied buffers
8
9      // place value into buffer
10     public synchronized void set( int value )
11     {
12         // for display, get name of thread that called this method
13         String name = Thread.currentThread().getName();
14

```

Figure 6.4 | *SynchronizedBuffer synchronizes access to a shared integer.*
(Part 1 of 3.)

```

15     // while no empty buffers, place thread in waiting state
16     while ( occupiedBuffers == 1 )
17     {
18         // output thread and buffer information, then wait
19         try
20         {
21             System.err.println(      name + " tries to write." );
22             displayState( "Buffer full. " + name + " waits." );
23             wait(); // wait until buffer is empty
24         } // end try
25
26         // if waiting thread interrupted, print stack trace
27         catch ( InterruptedException exception )
28         {
29             exception.printStackTrace();
30         } // end catch
31
32     } // end while
33
34     buffer = value; // set new buffer value
35
36     // indicate producer cannot store another value
37     // until consumer retrieves current buffer value
38     ++occupiedBuffers;
39
40     displayStateC name + " writes " + buffer );
41
42     notify(); // tell waiting thread to enter ready state
43 } // end method set; releases lock on SynchronizedBuffer
44
45 // return value from buffer
46 public synchronized int get()
47 {
48     // for display, get name of thread that called this method
49     String name = Thread.currentThread().getName();
50
51     // while no data to read, place thread in waiting state
52     while ( occupiedBuffers == 0 )
53     {
54         // output thread and buffer information, then wait
55         try
56         {
57             System.err.println( name + " tries to read." );
58             displayStateC "Buffer empty. " + name + " waits." );

```

Figure 6.4 | *SynchronizedBuffer* synchronizes access to a shared integer.
(Part 2 of 3.)

```

59         wait();// wait until buffer contains new values
60     } // end try
61
62     //if waiting thread interrupted, print stack trace
63     catch ( InterruptedException exception )
64     {
65         exception.printStackTrace();
66     } // end catch
67
68 } // end while
69
70 // indicate that producer can store another value
71 // because consumer just retrieved buffer value
72 --occupiedBuffers;
73
74 displayState( name + " reads " + buffer );
75
76 notify(); // tell waiting thread to become ready
77
78 return buffer;
79 } // end method get; releases lock on SynchronizedBuffer
80
81 // display current operation and buffer state
82 public void displayState( String operation )
83 {
84     StringBuffer outputLine = new StringBuffer( operation );
85     outputLine.setLength( 40 );
86     outputLine.append( buffer + "\t\t" + occupiedBuffers );
87     System.err.println( outputLine );
88     System.err.println();
89 } // end method displayState
90
91 } // end class SynchronizedBuffer

```

Figure 6.4 | *Synchronizedbuffer synchronizes access to a shared integer.*
(Part 3 of 3.)

When the Producer thread's run method invokes synchronized method set (from line 26 of Fig. 5.2), the thread attempts to acquire a lock on the SynchronizedBuffer monitor object. [Note: When we talk about "acquiring a lock" on a Java monitor, we mean "gaining mutually exclusive access to a monitor" in the generic discussion of monitors in Section 6.2, Monitors.] If the lock is available, the Producer thread acquires the lock. Then the loop in method set (lines 16-32 of Fig. 6.4) determines whether occupiedBuffers is equal to one. If so, buffer is full, so line 21 outputs a message indicating that the Producer thread is trying to write a value, and line 22 invokes method displayState (lines 81-89) to output another message indicating that the buffer is full and that the Producer thread is in the *waiting* state. Note that

method `displayState` is not declared `Synchronized` because it is called in method `main` (line 18 of Fig. 6.5) before the producer and consumer threads are created and thereafter only from inside `Synchronized` methods `get` and `set` (lines 10 and 46 of Fig. 6.4). Only one thread can execute inside the `SynchronizedBuffer`'s monitor at once and `displayState` is accessed only from inside the monitor, so mutually exclusive access is enforced without needing to declare `displayState` `Synchronized`.

Line 23 invokes method `wait` (inherited from `Object` by `SynchronizedBuffer`; all Java classes inherit from `Object` directly or indirectly) to place the thread that called method `set` (i.e., the Producer thread) in the *waiting* state for the `SynchronizedBuffer` object. The call to `wait` causes the calling thread to release the lock on the `SynchronizedBuffer` object. This is important because the thread cannot currently perform its task—and holding the lock would prevent other threads from accessing the object. This would result in deadlock, because the condition variable on which the first thread was waiting would never change. After the producer thread calls `wait` in line 23, another thread can attempt to acquire the `SynchronizedBuffer` object's lock and invoke the object's `set` or `get` methods—in particular, a consumer thread can now empty the buffer, eventually allowing the waiting producer to proceed.

The producer thread remains in the *waiting* state until it is notified by another thread that it may proceed—at which point the producer thread returns to the *ready* state and waits for a processor. When the producer thread returns to the *running* state, the thread implicitly attempts to reacquire the lock on the `SynchronizedBuffer` object. If the lock is available, the producer reacquires the lock and method `set` continues executing with the next statement after `wait`. Because `wait` is called in a loop (lines 15-32), the loop continuation condition is tested to determine whether the thread can proceed with its execution. If not, `wait` is invoked again; otherwise, method `set` continues with the next statement after the loop.

Line 34 in method `set` assigns the argument value to `buffer`. Line 38 increments `occupiedBuffers` to indicate that the buffer now contains a value (i.e., a consumer can read the value, but a producer cannot put another value there yet). Line 40 invokes method `displayState` to output a line to the console window indicating that the producer is writing a new value into the buffer. Line 42 invokes method `notify` (inherited from `Object`). If the consumer thread is waiting, it enters the *ready* state, where it can attempt its task again (as soon as the thread is assigned a processor). Method `notify` returns immediately and method `set` returns to its caller. [Note: Invoking method `notify` works correctly in this program because only one thread calls method `get` at any time (the consumer thread). Programs that have multiple threads waiting on a condition should invoke `notifyAll` to ensure that multiple threads receive notifications properly.] When method `set` returns, it implicitly releases the lock on the `SynchronizedBuffer` object.

Methods `get` and `set` are implemented similarly. When the consumer thread's `run` method invokes `synchronized` method `get` (from line 28 of Fig. 5.3), the thread attempts to acquire a lock on the `SynchronizedBuffer` object. When it acquires the

lock, the while loop (lines 51-68 of Fig. 6.4) determines whether `occupiedBuffers` is equal to 0. If so, the buffer is empty, so line 57 outputs a message indicating that the consumer thread is trying to read a value, and line 58 invokes method `displayState` to output another message indicating that the buffer is empty and that the consumer thread waits. Line 59 invokes method `wait` to place the consumer thread that called method `get` in the *waiting* state for the `SynchronizedBuffer` object. Again, the call to `wait` causes the calling thread to release the lock on the `SynchronizedBuffer` object, so another thread can attempt to acquire the lock and invoke the object's `set` or `get` method. If the lock on the `SynchronizedBuffer` is not available (e.g., if the `ProducerThread` has not yet returned from method `set`) the consumer thread is blocked until the lock becomes available.

The consumer thread object remains in the *waiting* state until it is notified by the producer thread that it may proceed—at which point the consumer thread returns to the *ready* state and waits a processor. When it returns to the *running* state, the thread implicitly attempts to reacquire the lock on the `SynchronizedBuffer` object. If the lock is available, the consumer thread reacquires it and method `get` continues executing with the next statement after the `wait`. Because `wait` is called in a loop (lines 51-68), the loop continuation condition is tested to determine whether the thread can proceed. If not, `wait` is invoked again; otherwise, method `get` continues with the next statement after the loop. Line 72 decrements `occupiedBuffers` to indicate that buffer is now empty, line 74 outputs a line to the console window indicating the value the consumer just read and line 76 invokes method `notify`. If the producer thread is waiting for the lock on this `SynchronizedBuffer` object, it enters the *ready* state. As soon as it is assigned a processor, the thread will attempt to reacquire the lock and continue performing its task. Method `notify` returns immediately, then method `get` returns the value of buffer to its caller (line 78). [Note: Again, invoking method `notify` works correctly in this program because only one thread calls method `set` at any time (the producer thread).] When method `get` returns, the lock on the `SynchronizedBuffer` object is released.

Class `SharedBufferTest2` (Fig. 6.5) is similar to class `SharedBufferTest` (Fig. 5.5). `SharedBufferTest2` contains method `main` (lines 6-27), which launches the application. Line 9 instantiates a shared `SynchronizedBuffer` and assigns its reference to `SynchronizedBuffer` variable `sharedLocation`. We use a `SynchronizedBuffer` variable rather than a `Buffer` variable, so that `main` can invoke `SynchronizedBuffer` method `displayState`, which is not declared in interface `Buffer`. The `SynchronizedBuffer` object stores the data that is shared between the producer and consumer threads. Lines 11-18 display the column heads for the output. Lines 21-22 create a `Producer` object and a `Consumer` object, respectively, and pass `sharedLocation` to each constructor, so each object is initialized with a reference to the same `SynchronizedBuffer`. Next, lines 24-25 invoke method `start` on the producer and consumer threads to place them in the *ready* state. This launches these threads and sets up the initial call to each thread's `run` method. Finally, method `main` terminates and the `main` thread of execution dies.

Study the three sample outputs in Fig. 6.5. Observe that every integer produced is consumed exactly once. The synchronization and the variable `occupiedBuffers` ensure that the producer and consumer can execute only when it is their turn. The producer must go first; the consumer must wait if the producer has not produced since the consumer last consumed; the producer must wait if the consumer has not yet consumed the value that the producer most recently produced. In the first and second sample outputs, notice the comments indicating when the producer and consumer must wait to perform their respective tasks. In the third sample output, notice that the producer and consumer were fortuitously able to perform their tasks without waiting.

Self Review

1. Why does the `displayState` method not need to be declared as synchronized?
2. (T/F) If the `notify` method is never called in the `get` method, the producer will never finish producing because it will be indefinitely postponed at line 23 of Fig. 6.4.

Ans: 1) Although `occupiedBuffers` is a shared resource, the only time that `displayState` is called is inside synchronized methods or when the program contains only one thread. 2) False. It is possible that the buffer will never fill, in which case the producer will never execute line 23. This is the case in the third output of Fig. 6.5.

```

1  // Fig. 6.5: SharedBufferTest2.Java
2  // SharedBufferTest2 creates producer and consumer threads.
3
4  public class SharedBufferTest2
5  {
6      public static void main( String [] args )
7      {
8          // create shared object used by threads
9          SynchronizedBuffer sharedLocation = new SynchronizedBuffer0;
10
11         // Display column heads for output
12         StringBuffer columnHeads =
13             new StringBuffer( "Operation" );
14         columnHeads.setLength( 40 );
15         columnHeads.appendC "Buffer\t\t\tOccupied Count" );
16         System.err.println( columnHeads );
17         System.err.println();
18         sharedLocation.displayStateC "Initial State" );
19
20         // create producer and consumer objects
21         Producer producer = new Producer( sharedLocation );
22         Consumer consumer = new Consumer( sharedLocation );
23
24         producer.start(); // start producer thread

```

Figure 6.5 | Threads modifying a shared object with synchronization. (Part 1 of 4.)

```

25         consumer.start(); // start consumer thread
26
27     } // end main
28
29 } // end class SharedBufferTest2

```

Sample Output 1:

Operation	Buffer	Occupied Count
Initial State	-1	0
Consumer tries to read. Buffer empty. Consumer waits.	-1	0
Producer writes 1	1	1
Consumer reads 1	1	0
Consumer tries to read. Buffer empty. Consumer waits.	1	0
Producer writes 2	2	1
Consumer reads 2	2	0
Producer writes 3	3	1
Consumer reads 3	3	0
Consumer tries to read. Buffer empty. Consumer waits.	3	0
Producer writes 4	4	1
Consumer reads 4	4	0
Producer done producing. Terminating Producer.		
Consumer read values totaling: 10. Terminating Consumer.		

Figure 6.5 | *Threads modifying a shared object with synchronization. (Part 2 of 4.)*

Sample Output 2:

Operation	Buffer	Occupied Count
Initial State	-1	0
Consumer tries to read. Buffer empty. Consumer waits.	-1	0
Producer writes 1	1	1
Consumer reads 1	1	0
Producer writes 2	2	1
Producer tries to write. Buffer full. Producer waits.	2	1
Consumer reads 2	2	0
Producer writes 3	3	1
Consumer reads 3	3	0
Producer writes 4	4	1
Producer done producing, Terminating Producer.	4	0
Consumer reads 4	4	0
Consumer read values totaling: 10. Terminating Consumer.		

Sample Output 3:

Operation	Buffer	Occupied Count
Initial State	-1	0
Producer writes 1	1	1

Figure 6.5 | Threads modifying a shared object with synchronization. (Part 3 of 4.)

Sample Output 3 (Cont.):

Operation	Buffer	Occupied Count
Initial State	-1	0
Producer writes 1	1	1
Consumer reads 1	1	0
Producer writes 2	2	1
Consumer reads 2	2	0
Producer writes 3	3	1
Consumer reads 3	3	0
Producer writes 4	4	1
Producer done producing. Terminating Producer.		
Consumer reads 4	4	0
Consumer read values totaling: 10. Terminating Consumer.		

Figure 6.5 | ~~Threads~~ modifying a shared object with synchronization (Part 4 of 4.)

6.5 Java Multithreading Case Study, Part IV:
Circular Buffer in Java

The program of Section 6.4 uses thread synchronization to guarantee that two threads manipulate data in a shared buffer correctly. However, the application may not perform optimally. If the two threads operate at different speeds, one of them will spend more (or even most) of its time waiting. If the producer thread produces values faster than the consumer can consume them, the producer will spend most of its time waiting for the consumer. Similarly, if the consumer consumes faster than the producer can produce them, the consumer will spend most of its time waiting for the producer. Even when the threads operate at the same relative speeds, over a period of time, occasionally those threads may become "out of sync," causing one of them to wait for the other. We cannot and should not make assumptions about the relative speeds of asynchronous concurrent threads. Too many interactions occur

with the operating system, the network, the user and other system components that . can cause the threads to operate at different and unpredictable speeds. When this happens in the producer/consumer example, one thread must wait. When threads spend a significant portion of their time waiting, programs may become less efficient, the system may become less responsive to interactive users and applications may suffer long delays because the processor is not used efficiently.

To minimize the waiting time for threads that share resources and operate at the same average speeds, we know from earlier in this chapter that we can implement a circular buffer that provides extra buffer slots into which the producer can place values at times when it is running faster than the consumer and from which the consumer can retrieve those values when it is running faster than the producer.

The key to using a circular buffer is to provide it with sufficient buffer slots to handle the anticipated "extra" production. If, over a period of time, we determine that the producer often produces as many as three more values than the consumer can consume, we can provide a buffer of three or more slots to handle the extra production. If the number of buffer slots is too small, threads will wait more; if too large, that would waste memory.

The Java program of Fig. 6.6-Fig. 6.7 demonstrates a producer and a consumer accessing a circular buffer (in this case, a shared array of three cells) with synchronization. In this version of the producer/consumer relationship, the consumer consumes a value only when the array is not empty and the producer produces a value only when the array is not full.

Class `Producer` is slightly modified from the version presented in Fig. 5.2; this version produces values from 11 to 20 (rather than 1-4). Class `Consumer` is slightly modified from the version in Fig. 5.3; this version consumes ten (rather than four) values from the circular buffer.

The significant changes of Fig. 6.4-Fig. 6.5 occur in `CircularBuffer` (Fig. 6.6), which replaces `SynchronizedBuffer` (Fig. 6.4). `CircularBuffer` contains four fields. `Array buffers` (line 8) implements the circular buffer as a three-element integer array. Variable `occupiedBuffers` (line 11) is the condition variable that can be used to determine whether a producer can write into the circular buffer (i.e., when `occupiedBuffers` is less than the number of elements in buffers) and whether a consumer can read from the circular buffer (i.e., when `occupiedBuffers` is greater than 0). Variable `readLocation` (line 14) indicates the position in buffers from which the next value can be read by a consumer. Variable `writeLocation` (line 15) indicates the next location in buffers in which a value can be placed by a producer.

```
1 // Fig, 6.6: CircularBuffer.Java
2 // CircularBuffer synchronizes access to an array of
3 // shared buffers.
```

Figure 6.6 | *SynchronizedBuffer controls a access to a shared array of integers.*
(Part 1 of 5.)

```

4
5 public class CircularBuffer implements Buffer
6 {
7     // each array element is a buffer
8     private int buffers[] = { -1, -1, -1 };
9
10    // occupiedBuffers maintains count of occupied buffers
11    private int occupiedBuffers = 0;
12
13    // variables that maintain read and write buffer locations
14    private int readLocation = 0;
15    private int writeLocation = 0;
16
17    // place value into buffer
18    public synchronized void set( int value )
19    {
20        // get name of thread that called this method
21        String name = Thread.currentThread().getName();
22
23        // while buffer full, place thread in waiting state
24        while ( occupiedBuffers == buffers.length )
25        {
26            // output thread and buffer information, then wait
27            try
28            {
29                System.err.println( "\nAll buffers full. " +
30                    name + " waits." );
31                wait(); // wait until space is available
32            } // end try
33
34            // if waiting thread interrupted, print stack trace
35            catch ( InterruptedException exception )
36            {
37                exception.printStackTrace();
38            } // end catch
39
40        } // end while
41
42        // place value in writeLocation of buffers
43        buffers[ writeLocation ] = value;
44
45        // output produced value
46        System.err.println( "\n" + name + " writes " +
47            buffers[ writeLocation ] + " " );
48

```

Figure 6.6 | *SynchronizedBuffer* controls access to a shared array of integers.
(Part 2 of 5.)

```

49     // indicate that one more buffer is occupied
50     ++occupiedBuffers;
51
52     // update writeLocation for future write operation
53     writeLocation = ( writeLocation + 1 ) % buffers.length;
54
55     // display contents of shared buffers
56     System.err.println( createStateOutput() );
57
58     notify(); // return a waiting thread to ready state
59 } // end method set
60
61 // return value from buffer
62 public synchronized int get()
63 {
64     // get name of thread that called this method
65     String name = Thread.currentThread().getName();
66
67     // while buffer is empty, place thread in waiting state
68     while ( occupiedBuffers == 0 )
69     {
70         // output thread and buffer information, then wait
71         try
72         {
73             System.err.println( "\nAll buffers empty. " +
74                 name + " waits." );
75             wait(); // wait until buffer contains new data
76         } // end try
77
78         // if waiting thread interrupted, print stack trace
79         catch ( InterruptedException exception )
80         {
81             exception.printStackTrace();
82         } // end catch
83
84     } // end while
85
86     // obtain value at current readLocation
87     int readValue = buffers[ readLocation ];
88
89     // output consumed value
90     System.err.println( "\n" + name + " reads " +
91         readValue + " " );
92
93     // decrement occupied buffers value

```

Figure 6.6 | SynchronizedBuffer controls access to a shared array of integers.
(Part 3 of 5.)

```

94         --occupiedBuffers;
95
96         // update readLocation for future read operation
97         readLocation = ( readLocation + 1 ) % buffers.length;
98
99         // display contents of shared buffers
100        System.err.println(    createStateOutput()    );
101
102        notify();    // return a waiting thread to ready state
103
104        return readValue;
105    } // end method get
106
107    // create state output
108    public String createStateOutput()
109    {
110        // first line of state information
111        String output = "(buffers occupied: " +
112            occupiedBuffers + ")\nbuffers: ";
113
114        for ( int i = 0; i < buffers.length; ++i )
115        {
116            output += " " + buffers[ i ] + " ";
117        } // end for
118
119        // second line of state information
120        output += "\n        ";
121
122        for ( int i = 0; i < buffers.length; ++i )
123        {
124            output += "---- ";
125        } // end for
126
127        // third line of state information
128        output += "\n        ";
129
130        // append readLocation (R) and writeLocation (W)
131        // indicators below appropriate buffer locations
132        for ( int i = 0; i < buffers.length; ++i )
133        {
134            if ( i == writeLocation &&
135                writeLocation == readLocation )
136            {
137                output += " WR ";
138            } // end if

```

*Figure 6.6 | SynchronizedBuffer controls access to a shared array of integers.
(Part 4 of 5.)*


```

139         else if ( i == writeLocation )
140         {
141             output += " W ";
142         } // end if
143         else if ( i == readLocation )
144         {
145             output += " R ";
146         } // end if
147         else
148         {
149             output += " ";
150         } // end else
151     } // end for
152
153     output += "\n";
154
155     return output;
156 } // end method createStateOutput
157
158
159 } // end class CircularBuffer

```

Figure 6.6 | *SynchronizedBuffer* controls access to a shared array of integers.
(Part 5 of 5.)

CircularBuffer method *set* (lines 17-59) performs the same tasks as did the *SynchronizedBuffer* in Fig. 6.4, with a few modifications. The *while* loop in lines 23-40 determines whether the producer must wait (i.e., all buffers are full). If so, lines 29-30 output that the producer is waiting to perform its task. Then line 31 invokes method *wait* to place the producer thread in the *waiting* state for the *CircularBuffer* object. When execution eventually continues in line 43 after the *while* loop, the value written by the producer is placed in the circular buffer at location *writeLocation*. Next, lines 46-47 output the value produced. Line 50 increments *occupiedBuffers*—there is now at least one value in the buffer that the consumer can read. Then, line 53 updates *writeLocation* for the next call to *CircularBuffer* method *set*. The output continues at line 56 by invoking method *createStateOutput* (lines 107-157), which outputs the number of occupied buffers, the contents of the buffers and the current *writeLocation* and *readLocation*. Finally, line 58 invokes method *notify* to indicate that the consumer thread waiting on the *CircularBuffer* object (if indeed the consumer is waiting) should proceed.

Method *get* (lines 61-105) of class *CircularBuffer* also performs the same tasks as it did in Fig. 6.4, with a few minor modifications. The *while* loop in lines 67-84 determines whether the consumer must wait (i.e., all buffer slots are empty). If the consumer thread must wait, lines 73-74 output that the consumer is waiting to perform its task. Then, line 75 invokes method *wait* to place the consumer thread in the *waiting* state for the *CircularBuffer* object. When execution eventually

continues at line 87 after the while loop, `readValue` is assigned the value at location `readLocation` in the circular buffer. Lines 90-91 output the consumed value. Line 94 decrements the `occupiedBuffers`—there is now at least one open position in the buffer in which the producer thread can place a value. Then, line 97 updates `readLocation` for the next call to `CircularBuffer` method `get`. Line 100 invokes method `createStateOutput` to output the number of occupied buffers, the contents of the buffers and the current `writeLocation` and `readLocation`. Finally, line 102 invokes method `notify` to indicate that the producer thread waiting for the `CircularBuffer` object (if indeed the producer is waiting) should proceed, and line 104 returns the consumed value to the calling method. Note that because Java implements signal-and-continue monitors, this program does not require the output parameter discussed in Section 6.2.3, Monitor Example: Circular Buffer.

Class `CircularBufferTest` (Fig. 6.7) contains the main method (lines 8–24) that launches the application. Line 13 creates the `CircularBuffer` object `sharedLocation`. Lines 19-20 create the producer and consumer threads, and lines 22-23 start them. The sample outputs include the current `occupiedBuffers`, the contents of the buffers and the current `writeLocation` and `readLocation`. In the output, the letters W and R represent the current `writeLocation` and `readLocation`, respectively. Notice that, after the third value is placed in the third element of the buffer, the fourth value is inserted back at the beginning of the array—this is the circular buffer effect.

```

1  // Fig. 6.7: CircularBufferTest.java
2  // CircularBufferTest shows two threads manipulating a
3  // circular buffer.
4
5  // set up the producer and consumer threads and start them
6  public class CircularBufferTest
7  {
8      public static void main ( String args[] )
9      {
10         // create shared object for threads; use a reference
11         // to a CircularBuffer rather than a Buffer reference
12         // to invoke CircularBuffer method createStateOutput
13         CircularBuffer sharedLocation = new CircularBuffer();
14
15         // display initial state of buffers in CircularBuffer
16         System.err.println( sharedLocation.createStateOutput() );
17
18         // set up threads
19         Producer producer = new Producer( sharedLocation );
20         Consumer consumer = new Consumer( sharedLocation );
21

```

Figure 6.7 | *CircularBufferTest* instantiates producer and consumer threads.
(Part 1 of 6.)

```

22     producer.start(); // start producer thread
23     consumer.start(); // start consumer thread
24 } // end main
25
26 } // end class CircularBufferTest

```

Sample Output:

```

(buffers occupied: 0)
buffers:  -1  -1  -1
-----
          WR

All buffers empty. Consumer waits.

Producer writes 11
(buffers occupied: 1)
buffers:  11  -1  -1
-----
          R  W

Consumer reads 11
(buffers occupied: 0)
buffers:  11  -1  -1
-----
          WR

Producer writes 12
(buffers occupied: 1)
buffers:  11  12  -1
-----
          R  W

Producer writes 13
(buffers occupied: 2)
buffers:  11  12  13
-----
          W  R

```

Figure 6.7 | *CircularBufferTest* instantiates producer and consumer threads.
(Part 2 of 6.)

Sample Output (Cont.):

```

Consumer reads 12
(buffers occupied: 1)
buffers:  11  12  13
-----
           W           R

Producer writes 14
(buffers occupied: 2)
buffers:  14  12  13
-----
           W           R

Producer writes 15
(buffers occupied: 3)
buffers:  14  15  13
-----
                        WR

All buffers full. Producer waits.

Consumer reads 13
(buffers occupied: 2)
buffers:  14  15  13
-----
           R           W

Producer writes 16
(buffers occupied: 3)
buffers:  14  15  16
-----
           WR

All buffers full. Producer waits.

```

Figure 6.7 | *CircularBufferTest* instantiates producer and consumer threads.
(Part 3 of 6.)

Sample Output (Cont.):

```

Consumer reads 14
(buffers occupied: 2)
buffers:  14   15   16
-----
           W     R

Producer writes 17
(buffers occupied: 3)
buffers:  17   15   16
-----
           WR

Consumer reads 15
(buffers occupied: 2)
buffers:  17   15   16
-----
           W     R

Consumer reads 16
(buffers occupied: 1)
buffers:  17   15   16
-----
           R     W

Consumer reads 17
(buffers occupied: 0)
buffers:  17   15   16
-----
           WR

Producer writes 18
(buffers occupied: 1)
buffers:  17   18   16
-----
           R     W

```

Figure 6.7 | CircularBufferTest instantiates producer and consumer threads.
(Part 4 of 6.)

Sample Output (Cont.):

```
Consumer reads 18
(buffers occupied: 0)
buffers: 17 18 16
```

```
-----
```

```
WR
```

All buffers empty. Consumer waits.

```
Producer writes 19
(buffers occupied: 1)
buffers: 17 18 19
```

```
-----
```

```
W
```

```
R
```

```
Consumer reads 19
(buffers occupied: 0)
buffers: 17 18 19
```

```
-----
```

```
WR
```

```
Producer writes 20
(buffers occupied: 1)
buffers: 20 18 19
```

```
-----
```

```
R
```

```
W
```

Producer done producing.

Terminating Producer.

```
Consumer reads 20
(buffers occupied: 0)
buffers: 20 18 19
```

```
-----
```

```
WR
```

Figure 6.7 | *CircularBufferTest* instantiates producer and consumer threads.
(Part 5 of 6.)

Sample Output (Cont.):

```
Consumer read values totaling: 155.
Terminating Consumer.
```

Figure 6.7 | *CircularBufferTest* instantiates producer and consumer threads.
(Part 6 of 6.)

Self Review

1. What are the potential costs and benefits of making the circular buffer larger?
- 1 What would happen if line 102 of Fig. 6.6 was omitted?

Ans: **1)** The potential benefit is that the producer can produce more and block less if the consumer is temporarily slower than the producer. However, this could result in wasted memory if both the consumer and producer work at the same speed, if one works much faster than the other or if the producer rarely, if ever, gets far enough ahead of the consumer to take advantage of the extra space. **2)** The consumer thread would complete without issuing the notify. If the producer fortuitously never waits for the consumer (indeed a rare occurrence), then the system would function correctly. More commonly, however, if the producer were waiting it would never be able to proceed, i.e., the producer could be deadlocked.

Web Resources

developer.java.sun.com/developer/Books/performance2/chap4.pdf

This page from Sun's Web site provides a detailed description of how to use Java monitors.

hissa.nist.gov/rbac/5277/titlerpc.html

Compares several popular RPC implementations.

java.sun.com/docs/books/tutorial/essential/threads/synchronization.html

This is Sun's example of implementing mutual exclusion in the consumer/producer relationship in Java.

Summary

Concurrent programs are harder to write, debug, modify, and prove correct than non-concurrent programs. Nonetheless, there has been a surge of interest in concurrent programming languages because they enable us to express more naturally solutions to problems that are inherently parallel. The proliferation of multiprocessing systems, distributed systems and massively parallel architectures has also fueled the surge.

A monitor is an object that contains both the data and procedures needed to perform allocation one or more serially reusable shared resources. Monitor data is accessible only within the monitor; there is no way for threads outside the monitor to access monitor data directly. To accomplish a resource allocation function using monitors, a

thread must call a monitor entry routine. Many threads might want to enter the monitor at various times, but mutual exclusion is rigidly enforced at the monitor boundary. A thread that tries to enter the monitor when it is in use is made to wait by the monitor.

Eventually, the thread that has the resource will call a monitor entry routine to return the resource. There might be threads waiting for the resource, so the monitor entry routine calls signal to allow one of the waiting threads to acquire the resource and enter the monitor. To avoid indefinite postponement, a monitor gives higher priority to waiting threads than to newly arrived threads.

Before a thread can reenter the monitor, the thread calling signal must first exit the monitor. A signal-and-exit

monitor requires a thread to immediately exit the monitor upon signaling. Alternatively, a signal-and-continue monitor allows a thread inside the monitor to signal that the monitor will soon become available, but still maintain a lock on the monitor until the thread exits the monitor. A thread can exit the monitor by waiting on a condition variable or by completing execution of the code protected by the monitor.

A thread currently inside a monitor may need to wait outside the monitor until another thread performs an action inside the monitor. A monitor associates a separate condition variable with each distinct situation that might cause a thread to have to wait. Every condition variable has an associated queue. A thread calling `wait` on a particular condition variable is placed into the queue associated with that condition variable; a thread calling `signal` on a particular condition variable causes a thread waiting on that condition variable (if there is such a thread) to be removed from the queue associated with that condition variable and to enter the monitor.

In the circular buffer implementation of the solution to the producer/consumer problem, the producer deposits data in the successive elements of the shared array. The consumer removes them in the order in which they were deposited (FIFO). The producer can be several items ahead of the consumer. Eventually, the producer fills the last element of the array. When it produces more data, it must "wrap around" and again begin depositing data in the first element of the array.

Because of the fixed size of the circular buffer, the producer will occasionally find all the array elements full; in this case the producer must wait until the consumer empties an array element. Similarly, there will be times when the consumer wants to consume, but the array will be empty; in this case the consumer must wait until the producer deposits data into an array element.

If the two threads operate at different speeds, one of them will spend more (or even most) of its time waiting. If the producer thread produces values faster than the consumer can consume them, the producer thread spends most of its time waiting for the consumer to remove the next value from the array. Similarly, if the consumer thread consumes values faster than the producer can produce them, the consumer thread spends most of its time waiting for the producer to place the next value into the array.

Key Terms

bounded buffer—See circular buffer.

In computer systems, it is common to have some threads (called readers) that read data and others (called writers) that write it. Because readers do not change the contents of the database, many readers can access the database at once. But a writer can modify the data, so it must have exclusive access. A new reader can proceed as long as no thread is writing and no writer thread is waiting to write. Each new reader signals the next waiting reader to proceed. This causes a "chain reaction" that continues until all waiting readers have become active. While this chaining is in progress, all arriving threads are forced to wait. During the chaining, arriving readers cannot enter the monitor, because the monitor observes the rule that signaled threads are serviced before arriving threads. If arriving readers were allowed to proceed, then a continuous stream of arriving readers would indefinitely postpone waiting writers. When the last reader leaves the monitor, the thread signals a waiting writer to proceed. When a writer finishes, it first checks if there is a waiting reader. If there is, the waiting reader proceeds (again cascading in all waiting readers). If there is no waiting reader, a waiting writer is allowed to proceed.

Monitors are the primary mechanism providing mutual exclusion and synchronization in multithreaded Java applications. The keyword `synchronized` imposes mutual exclusion on an object in Java. Java monitors are signal-and-continue monitors, allowing a thread to signal that the monitor will soon become available, but still maintain a lock on the monitor until the thread exits the monitor.

In Java, the `wait` method causes the calling thread to release the lock on the monitor and wait on an unnamed condition variable. After calling `wait`, a thread is placed in the wait set. A thread remains in the wait set until signaled by another thread. Because the condition variable is implicit in Java, a thread may be signaled, reenter the monitor and find that the condition on which it waited has not been met.

Threads issue signals by calling method `notify` or `notifyAll`. The `notify` method wakes a thread in the wait set. If more than two threads may access a monitor, it is best to use the `notifyAll` method, which wakes all threads in the wait set. Because `notifyAll` wakes all threads attempting to enter the monitor (instead of a single thread), `notify` can yield higher performance in some applications.

circular buffer—In the producer/consumer relationship, a fixed-size region of shared memory that stores multiple

values produced by a producer. If the producer occasionally produces values faster than the consumer, a circular buffer reduces the time the producer spends waiting for a consumer to consume the values, when compared to a buffer that stores a single value. If the consumer temporarily consumes values faster than the producer, a circular buffer can similarly reduce the time a consumer spends waiting for the producer to produce values.

condition variable—Variable that contains a value and an associated queue. When a thread waits on a condition variable inside a monitor, it exits the monitor and is placed in the condition variable's queue. Threads wait in the queue until signaled by another thread.

entry queue—See entry set.

entry set—In Java, a queue of threads waiting to enter a monitor after calling a synchronized method.

information hiding—Software architectural technique that facilitates the development of more reliable software systems by preventing direct access to data within an object by outside objects.

monitor—Concurrency construct that contains both the data and procedures needed to provide mutual exclusion while allocating a serially reusable shared resource or group of serially reusable shared resources.

monitor entry routine—Monitor routine that can be called by any thread, but that can be executed by only one thread at a time. Unlike private monitor routines, which can be called only by threads executing inside the monitor, monitor entry routines enforce mutual exclusion.

notify—Java method that wakes one thread in a monitor's wait set. The thread that is awakened depends on the JVM implementation.

notifyAll—Java method that awakens all threads in a monitor's wait and entry sets. Method `notifyAll` ensures that waiting threads are not indefinitely postponed, but incurs more overhead than `notify`.

serially reusable shared resource—Resource that can be used by only one thread at a time.

signal-and-continue monitor—Monitor that allows a thread to signal that the monitor is available, but does not require the thread to release the lock until it exits the monitor, at which point a signaled thread may enter the monitor.

signal-and-exit monitor—Monitor that requires a thread to release the lock on the monitor as soon as the thread signals another thread.

Solo—Small operating system created by Per Brinch Hansen to demonstrate fail-safe concurrent programming.

synchronization—Coordination between asynchronous concurrent threads to sequentialize their access to shared resources.

synchronized—Java keyword that imposes mutual exclusive access to code inside an object.

wait queue—See wait set.

wait set—In Java, a set of threads waiting to reacquire the lock on a monitor.

Exercises

6.1 Compare and contrast the use of monitors and semaphore operations.

6.2 When a resource is returned by a thread calling a monitor, the monitor gives priority to a waiting thread over a new requesting thread. Why?

6.3 How do condition variables differ from conventional variables? Does it make sense to initialize condition variables?

6.4 The text has stated repeatedly that no assumptions should be made about the relative speeds of asynchronous concurrent threads. Why?

6.5 What factors, do you suppose, would affect a designer's choice of the number of slots a circular buffer should have?

6.6 Why is it considerably more difficult to test, debug, and prove program correctness for concurrent programs than for sequential programs?

6.7 The text states that information hiding is a system structuring technique that contributes to the development of more reliable software systems. Why, do you suppose, is this so?

6.8 Refer to the monitor described in Fig. 6.2 and answer each of the following questions.

- Which procedure places data into the circular buffer?
- Which procedure removes data from the circular buffer?
- Which queuing discipline best describes the operation of the circular buffer?

- d. Is this true: `writerPosition >= readerPosition`?
- e. Which statements perform monitor initialization?
- f. Which statement(s) can "wake up" a thread waiting on a condition variable?
- g. Which statement(s) can put a thread "to sleep"?
- h. Which statement(s) ensure that the buffer "wraps around"?
- i. Which statement(s) modify a shared critical variable to indicate that another slot in the buffer is available?

6.9 In the readers and writers monitor presented in Fig. 6.3 why does it make sense to cascade in all waiting readers? Could this cause indefinite postponement of waiting writers? Under what circumstances might you choose to limit the number of waiting readers you would initiate when reading is allowed?

6.10 (*The Sleeping Barber Problem*)⁵³ A barbershop has a cutting room with one chair and a waiting room with n chairs. Customers enter the waiting room one at a time if space is available, otherwise they go to another shop. Each time the barber finishes a haircut the customer leaves to go to another store, and a waiting customer, if there is one, enters the cutting room and has a haircut. Customers may enter the waiting room one at a time, or waiting customers may enter the (empty) cutting room one at a time, but these events are mutually exclusive. If the barber discovers that the waiting room is empty, the barber falls asleep in the waiting room. An arriving customer, finding the barber asleep, wakes the barber and has a haircut; otherwise the arriving customer waits. Use a monitor to coordinate the operation of the barber and the clients. If you know Java, also implement your monitor in Java.

6.11 (*The Cigarette Smokers Problem*)⁵⁴ [Note: One of the authors, HMD, worked with S. Patil and Jack Dennis in the Computation Structures Group at M.I.T.'s Project Mac] This

has become one of the classic problems in concurrency control. Three smokers are represented by threads S1, S2, and S3. Three vendors are represented by threads V1, V2, and V3. Each smoker requires tobacco, a wrapper, and a match to smoke; when these resources are available, the smoker smokes the cigarette to completion, then becomes eligible to smoke again. S1 has tobacco, S2 has wrappers, and S3 has matches. V1 supplies tobacco and wrappers, V2 supplies wrappers and matches, and V3 supplies matches and tobacco. V1, V2, and V3 operate in mutual exclusion; only one of these threads can operate at a time and the next vendor cannot operate until resources supplied by the previous vendor have been consumed by a smoker. Use a monitor to coordinate the operation of the smoker and vendor threads.

6.12 Semaphores are at least as powerful as monitors. Show how to implement a monitor by using semaphores.

6.13 Use semaphores to solve the readers and writers problem.

6.14 Implement the readers and writers problem. Model your solution after the readers and writers solution in Fig. 6.2 and the Java-based circular-buffer solution in Fig. 6.6 and Fig. 6.7.

6.15 Should a waiting thread receive priority over a thread first attempting to enter a monitor? What priority scheme, if any, should be imposed on waiting threads?

Suggested Project

6.16 Prepare a research paper on concurrent programming in video game consoles. Does the hardware provide mutual exclusion primitives to the application programmer?

Suggested Simulation

6.17 Extend the producer/consumer solution without a circular buffer presented in Fig. 6.1 to handle multiple producers.

Recommended Reading

Solutions to problems in concurrent programming have been known for quite some time. Dijkstra⁵⁵ was the first to truly address such concerns, and he was soon followed by many others.^{56, 57, 58, 59} Recent developments in mutual exclusion algorithms pertain to distributed systems and are beyond the scope of this chapter. However, the reader is encouraged to study Lamport's survey of the mutual exclusion problem and some powerful results.^{60, 61} Initial development of the concept of a

monitor has been attributed to Brinch Hansen⁶² and Hoare.⁶³ An excellent survey of monitors, which describes their history, taxonomy and usage, is presented by Buhr.⁶⁴ For more detailed information about the Java programming language and concurrent programming, see *Java How to Program*⁶⁵ and *Concurrent Programming in Java: Design Principles and Pattern*, 2nd ed.⁶⁶ The bibliography for this chapter is located on our Web site at www.deitel.com/books/os3e/Bibliography.pdf.

Works Cited

1. "Rationale for the Design of the Ada Programming Language," *ACM SIGPLAN Notices*, Vol. 14, No. 6, June 1979, Part B.
2. Carvalho, O. S. F., and G. Roucairol, "On Mutual Exclusion in Computer Networks," *Communications of the ACM*, Vol. 26, No. 2, February 1983, pp. 146-147.
3. Liskov, B., and R. Scheifler, "Guardians and Actions: Linguistic Support for Robust, Distributed Programs," *ACM Transactions on Programming Languages and Systems*, Vol. 5, No. 3, 1983, pp. 381-404.
4. Shatz, S. M., "Communication Mechanisms for Programming Distributed Systems," *Computer*, Vol. 17, No. 6, June 1984, pp. 21-28.
5. Fisher, D. A., and R. M. Weatherly, "Issues in the Design of a Distributed Operating System for Ada," *Computer*, Vol. 19, No. 5, May 1986, pp. 38-47.
6. Liskov, B. H.; M. Herlihy; and L. Gilbert, "Limitations of Synchronous Communication with Static Process Structure in Languages for Distributed Computing," *Proceedings of the 13th ACM Symposium on Principles of Programming Languages*, St. Petersburg, Florida, January 1986.
7. Shatz, S. M., and J. Wang, "Introduction to Distributed-Software Engineering," *Computer*, Vol. 20, No. 10, October 1987, pp. 23-32.
8. Roberts, E. S.; A. Evans, Jr.; C. R. Morgan; and E. M. Clarke, "Task Management in Ada—A Critical Evaluation for Real-Time Multiprocessors," *Software—Practice and Experience*, Vol. 11, No. 10, October 1981, pp. 1019-1051.
9. Nielsen, K. W., and K. Shumate, "Designing Large Real-Time Systems with Ada," *Communications of the ACM*, Vol. 30, No. 8, August 1987, pp. 695-715.
10. Ford, R., "Concurrent Algorithms for Real-Time Memory Management," *IEEE Software*, Vol. 5, No. 5, September 1988, pp. 10-24.
11. "Preliminary Ada Reference Manual," *ACM SIGPLAN Notices*, Vol. 14, No. 6, June 1979, Part A.
12. Brinch Hansen, P., "The Programming Language Concurrent Pascal," *IEEE Transactions on Software Engineering*, Vol. SE-1, No. 2, June 1975, pp. 199-207.
13. Coleman, D; Gallimore, R. M.; Hughes, J. W.; and M. S. Powell, "An Assessment of Concurrent Pascal," *Software—Practice and Experience*, Vol. 9, 1979, pp. 827-837.
14. Brinch Hansen, P., "Distributed Processes: A Concurrent Programming Concept," *Communications of the ACM*, Vol. 21, No. 11, November 1978, pp. 934-941.
15. Gehani, N. H., and W. D. Roome, "Concurrent C," *Software—Practice and Experience*, Vol. 16, No. 9, 1986, pp. 821-844.
16. Kieburtz, R. B., and A. Silberschatz, "Comments on 'Communicating Sequential Processes,'" *ACM Transactions on Programming Languages and Systems*, Vol. 1, No. 2, 1979, pp. 218-225.
17. Hoare, C. A. R., *Communicating Sequential Processes*, Englewood Cliffs, NJ: Prentice-Hall, 1985.
18. Hoppe, J., "A Simple Nucleus Written in Modula-2: A Case Study," *Software—Practice and Experience*, Vol. 10, No. 9, September 1980, pp. 697-706.
19. Wirth, R., *Programming in Modula-2*, New York: Springer-Verlag, 1982.
20. Ogilvie, J. W. L., *Modula-2 Programming*, New York: McGraw-Hill, 1985.
21. McGraw, J. R., "The VAL Language: Description and Analysis," *ACM Transactions on Programming Languages*, Vol. 4, No. 1, January 1982, pp. 44-82.
22. Cook, R. P., "MOD—A Language for Distributed Programming," *IEEE Transactions on Software Engineering*, Vol. SE-6, No. 6, 1980, pp. 563-571.
23. Dijkstra, E. W., "Hierarchical Ordering of Sequential Processes," *Acta Informatica*, Vol. 1, 1971, pp. 115-138.
24. Brinch Hansen, P., "Structured Multiprogramming," *Communications of the ACM*, Vol. 15, No. 7, July 1972, pp. 574-578.
25. Brinch Hansen, P., *Operating System Principles*, Englewood Cliffs, N.J.: Prentice Hall, 1973.
26. Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, Vol. 17, No. 10, October 1974, pp. 549-557. Corrigendum, *Communications of the ACM*, Vol. 18, No. 2, February 1975, p. 95.
27. Brinch Hansen, P., "The Solo Operating System: Processes, Monitors, and Classes," *Software—Practice and Experience*, Vol. 6, 1976, pp. 165-200.
28. Howard, J. H., "Proving Monitors," *Communications of the ACM*, Vol. 19, No. 5, May 1976, pp. 273-279.
29. Howard, J. H., "Signaling in Monitors," *Second International Conference on Software Engineering*, San Francisco, October 1976, pp. 47-52.
30. Lister, A. M., and K. J. Maynard, "An Implementation of Monitors," *Software—Practice and Experience*, Vol. 6, No. 3, July 1976, pp. 377-386.
31. Kessels, J. L. W., "An Alternative to Event Queues for Synchronization in Monitors," *Communications of the ACM*, Vol. 20, No. 7, July 1977, pp. 500-503.
32. Keedy, I., "On Structuring Operating Systems with Monitors," *Australian Computer Journal*, Vol. 10, No. 1, February 1978, pp. 23-27, reprinted in *Operating Systems Review*, Vol. 13, No. 1, January 1979, pp. 5-9.
33. Lampson, B. W., and D. D. Redell, "Experience with Processes and Monitors in MESA," *Communications of the ACM*, Vol. 23, No. 2, February 1980, pp. 105-117.
34. Wegner, P., and S. A. Smolka, "Processes, Tasks and Monitors: A Comparative Study of Concurrent Programming Primitives," *IEEE Transactions on Software Engineering*, Vol. SE-9, No. 4, July 1983, pp. 446-462.

286 Concurrent Programming

35. Buhr, P. A.; Fortier, M.; and Coffin, M., "Monitor Classifications," *ACM Computing Surveys*, Vol. 27, No. 1, March 1995, pp. 63-107.
36. Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, Vol. 17, No. 10, October 1974, pp. 549-557.
37. Buhr, P. A.; Fortier, M.; and Coffin, M., "Monitor Classification," *ACM Computing Surveys*, Vol. 27, No. 1, March 1995, pp. 63-107.
38. Brinch Hansen, P., "The Programming Language Concurrent Pascal," *IEEE Transactions on Software Engineering*, No. 2, June 1975, pp. 199-206.
39. Syracuse University, "Biography," <web.syr.edu/~pbhansen/html/biography.html>.
40. Syracuse University, "Biography," <web.syr.edu/~pbhansen/html/biography.html>.
41. Brinch Hansen, P., *Operating System Principles*, Englewood Cliffs, NJ: Prentice Hall, 1973.
42. Brinch Hansen, P., "The Purpose of Concurrent Pascal," *Proceedings of the International Conference on Reliable Software*, pp. 305-309, 1975.
43. Syracuse University, "Biography," <web.syr.edu/~pbhansen/html/biography.html>.
44. Brinch Hansen, P., "Java's Insecure Parallelism," *SIGPLAN Notices*, Vol. 34, No. 4, pp. 38-45, April 1999.
45. Brinch Hansen, P., "The Solo Operating System," *Software—Practice and Experience*, Vol. 6, No. 2, pp. 141-205, April-June 1976.
46. Brinch Hansen, P., "Joyce—A Programming Language for Distributed Systems," *Software—Practice and Experience*, Vol. 17, No. 1, January 1987, pp. 29-50.
47. Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, Vol. 17, No. 10, October 1974, pp. 549-557. Corrigendum, *Communications of the ACM*, Vol. 18, No. 2, February 1975, p. 95.
48. Courtois, P. X.; Heymans, F.; and D. L. Parnas, "Concurrent Control with Readers and Writers," *Communications of the ACM*, Vol. 14, No. 10, October 1971, pp. 667-668.
49. Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, Vol. 17, No. 10, October 1974, pp. 549-557. Corrigendum, *Communications of the ACM*, Vol. 18, No. 2, February 1975, p. 95.
50. Hartley, S. X., "Concurrent Programming Using the Java Language," Modified: 30 December 1997, <www.mcs.drexel.edu/~shartley/ConcProglava/monitors.html>.
51. Venners, B. "Thread Synchronization", Chapter 20 of *Inside the Java Virtual Machine*, last updated April 18, 2003, <www.artima.com/insidejvm/ed2/threadsynch.html>.
52. Christopher, T. W., and G. K. Thiruvathukal, *High-Performance Java Platform Computing, Multithreaded and Networked Programming*, Upper Saddle River, NJ: Prentice Hall PTR, February 2001, pp. 89-93.
53. Dijkstra, E. W., "Solution of a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 8, No. 5, September 1965, p. 569.
54. Patil, S. S., "Limitations and Capabilities of Dijkstra's Semaphores Primitives for Coordination among Processes," *M.I.T. Project MAC Computational Structures Group Memo 57*, February 1971.
55. Dijkstra, E. W., "Hierarchical Ordering of Sequential Processes," *Acta Informatica*, Vol. 1, 1971, pp. 115-138.
56. Knuth, D., "Additional Comments on a Problem in Concurrent Programming Control," *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 321-322.
57. Bernstein, A. J. "Program Analysis for Parallel Processing," *IEEE Transactions on Computers*, Vol. 15, No. 5, October 1966, pp. 757-762.
58. Courtois, P. X.; Heymans, E.; and D. L. Parnas, "Concurrent Control with Readers and Writers," *Communications of the ACM*, Vol. 14, No. 10, October 1971, pp. 667-668.
59. Lamport, L., "Concurrent Reading and Writing," *Communications of the ACM*, Vol. 20, No. 11, November 1977, pp. 806-811.
60. Lamport, L., "The Mutual Exclusion Problem: Part I—A Theory of Interprocess Communication," *Journal of the ACM (JACM)*, Vol. 33, No. 2, April 1986, pp. 313-326.
61. Lamport, L., "The Mutual Exclusion Problem: Part II—Statement and Solutions," *Journal of the ACM (JACM)*, Vol. 33, No. 2, April 1986, pp. 327-348.
62. Brinch Hansen, P., "Structured Multiprogramming," *Communications of the ACM*, Vol. 15, No. 7, July 1972, pp. 574-578.
63. Hoare, C. A. R., "Monitors: An Operating System Structuring Concept," *Communications of the ACM*, Vol. 17, No. 10, October 1974, pp. 549-557. Corrigendum, *Communications of the ACM*, Vol. 18, No. 2, February 1975, p. 95.
64. Buhr, P. A.; Fortier, M.; and M. Coffin, "Monitor Classification," *ACM Computing Surveys*, Vol. 27, No. 1, March 1995, pp. 63-107.
65. Deitel, H. M. and P. J. Deitel, *Java How to Program*, 5th ed., Upper Saddle River, NJ: Prentice Hall, 2003.
66. Lea, D., *Concurrent Programming in Java: Design Principles and Pattern*, 2nd ed., Reading, MA: Addison-Wesley, 1999.

We had better wait and see.

-H.H.Asquith-

Every man is the center of a circle, whose fatal circumference he cannot pass.

—John James Ingalls—

Hold me but safe within the bond

Of one immortal look.

— Robert Browning —

Detection is, or ought to be, an exact science, and should be treated in the same cold and Unemotional manner.

— Sir Arthur Conan Doyle —

Delays have dangerous ends.

—William Shakespeare—

Chapter 7

Deadlock and Indefinite Postponement

Objectives

After reading this chapter, you should understand:

- *the problem of deadlock.*
- *the four necessary conditions for deadlock to exist.*
- *the problem of indefinite postponement.*
- *the notions of deadlock prevention, avoidance, detection and recovery.*
- *algorithms for deadlock avoidance and detection.*
- *how systems can recover from deadlocks.*

Chapter Outline

7.1

Introduction

7.2

Examples of Deadlock

7.2.1 Traffic Deadlock

Anecdote: One-lane Bridge

7.2.2 Simple Resource Deadlock

Operating Systems Thinking: *Waiting, Deadlock and Indefinite Postponement*

7.2.3 Deadlock in Spooling Systems

7.2.4 Example: Dining Philosophers

7.3

Related Problem: Indefinite Postponement

7.4

Resource Concepts

7.5

Four Necessary Conditions for Deadlock

7.6

Deadlock Solutions

7.7

Deadlock Prevention

Anecdote: No-Nuts, Bolts or Screws Allowed

7.7.1 Denying the "Wait-For" Condition

7.7.2 Denying the "No-Preemption" Condition

7.7.3 Denying the "Circular-Wait" Condition

7.8

Deadlock Avoidance with Dijkstra's Bankers Algorithm

Anecdote: Acronyms

7.8.1 Example of a Safe State

7.8.2 Example of an Unsafe State

7.8.3 Example of Safe-State-to-Unsafe-State Transition

7.8.4 Banker's Algorithm Resource Allocation

7.8.5 Weaknesses in the Banker's Algorithm

7.9

Deadlock Detection

7.9.1 Resource-Allocation Graphs

7.9.2 Reduction of Resource-Allocation Graphs

7.10

Deadlock Recovery

7.11

Deadlock Strategies in Current and Future Systems

Web Resources | Summary | Key Terms | Exercises | Recommended Reading | Works Cited

7.1 Introduction

In the previous four chapters, we discussed asynchronous, concurrent processes and threads. A multiprogrammed system provides many benefits, but as discussed in

Chapter 6, Concurrent Programming, multiprogramming also introduces additional complexity. One problem that arises in multiprogrammed systems is **deadlock**. A process or thread is in a state of deadlock (or is **deadlocked**) if it is waiting for a particular event that will not occur. In a system deadlock, one or more processes are deadlocked.^{1,2} The remainder of this chapter will focus on processes, but most of the discussion applies to threads as well.

In multiprogrammed computing systems, resource sharing is one of the primary goals. When resources are shared among a set of processes, each process maintaining exclusive control over particular resources allocated to it, deadlocks can develop in which some processes will never be able to complete execution. The result can be loss of work and reduced system throughput and system failure.

This chapter discusses the problem of deadlock and summarizes the four major areas of deadlock research, namely **deadlock prevention, avoidance, detection and recovery**. It also considers the closely related problem of indefinite postponement, also called **starvation**, in which a process that is not deadlocked could wait for an event that might never occur or might occur unpredictably far in the future because of biases in the systems resource-scheduling policies. In some cases, the price for making a system free of deadlock is high. For some systems, such as those that are mission critical, the price must be paid no matter how high, because allowing a deadlock to develop could be catastrophic, especially if it puts human life at risk. This chapter also discusses solutions to the problems of deadlock and indefinite postponement in terms of trade-offs between their overhead and their anticipated benefits.

Self Review

1. A system designer decides to try to avoid any possibility of deadlock by creating a multiprogrammed system that does not share resources. What is wrong with this idea?
2. Compare and contrast deadlock with indefinite postponement.

Ans: 1) It would be highly inefficient, because each process would need its own set of resources. Also, asynchronous concurrent programs often require shared resources such as semaphores. **2)** Deadlock and indefinite postponement are similar in that they occur when processes wait for an event. Deadlock occurs because the event will never occur; indefinite postponement occurs because it is uncertain when or even whether the event will ever occur (due to biases in a system's resource-scheduling policies).

7.2 Examples of Deadlock

Deadlocks can develop in many ways. If a process is given the task of waiting for an event to occur, and if the system includes no provision for signaling that event, then we have a one-process deadlock.³ Such deadlocks are extremely difficult to detect. Deadlocks in real systems often involve multiple processes competing for multiple resources of multiple types. Let us consider several common examples.

7.2.1 Traffic Deadlock

Figure 7.1 illustrates a kind of deadlock that occasionally develops in cities. A number of automobiles are attempting to drive through a busy neighborhood, and the traffic is completely snarled. The police have to unwind the jam by slowly and carefully backing cars out of the congested area. Eventually the traffic begins to flow normally, but not without much annoyance, effort, and loss of time (see the Anecdote, One-Lane Bridge).

Self Review

1. Assuming that there are no cars beyond the ellipses in Fig. 7.1, what minimum number of cars would have to back up to relieve the deadlock and which car(s) would they be?
2. If cars could be removed by airlifting in Fig. 7.1, what minimum number of cars, and which one(s), would have to be removed to relieve the deadlock?

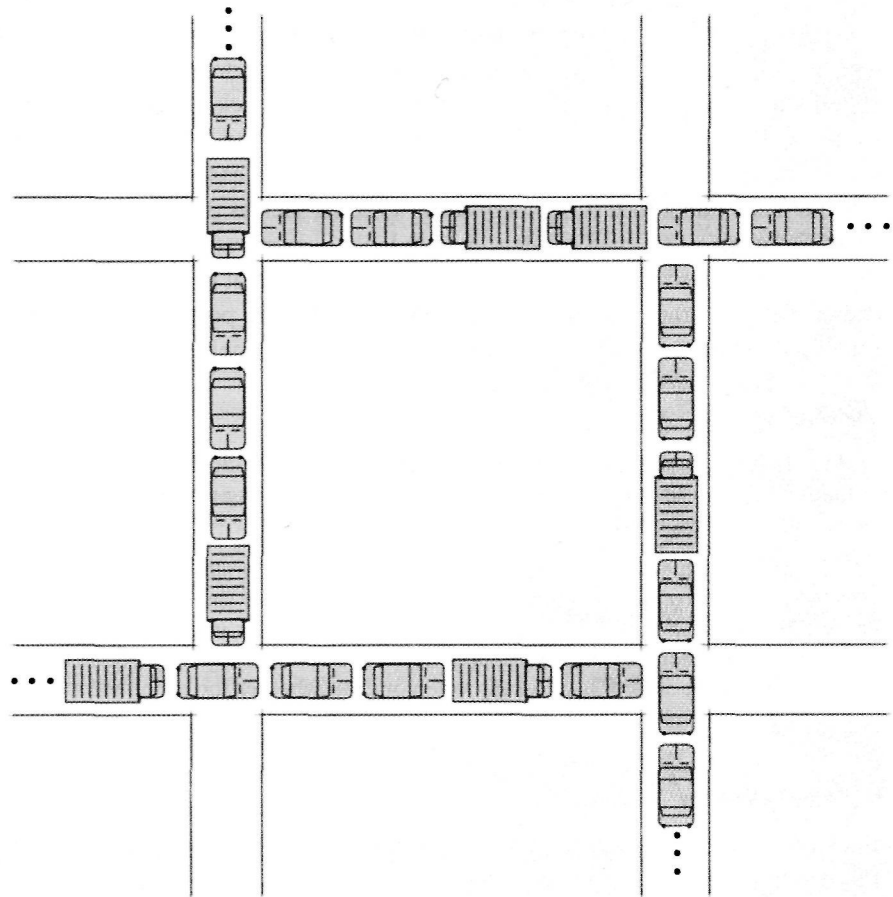


Figure 7.1 | Traffic deadlock example.⁴

Ans: 1.) In Fig. 7.1, only two cars would need to back up to allow every other car to eventually move—any one of the cars abutting an ellipsis, then the car ahead of that one in the intersection. **2.)** Only one car has to be removed—namely, any one of the four cars in the intersections.

7.2.2 Simple Resource Deadlock

Most deadlocks in operating systems develop because of the normal contention for **dedicated resources** (i.e., resources that may be used over time by many processes but by only one process at a time, sometimes called **serially reusable resources**). For example, a printer can print jobs for only one process at a time (otherwise pieces of many print jobs would be interwoven). A simple example of a resource deadlock is illustrated in Fig. 7.2. This **resource allocation graph** shows two processes as rectangles and two resources as circles. An arrow from a resource to a process indicates that the resource belongs to, or has been allocated to, the process. An arrow from a process to a resource indicates that the process is requesting, but has not yet been allocated, the resource. The diagram illustrates a deadlocked system: Process P_1 holds resource R_1 and needs resource R_2 to continue. Process P_2 holds resource R_2 and needs resource R_1 to continue. Each process is waiting for the other to free a resource that the other process will not free. This **circular wait** is characteristic of deadlocked systems (see the Operating Systems Thinking feature, Waiting, Deadlock and Indefinite Postponement). Holding resources tenaciously in this way, a is sometimes referred to as a **deadly embrace**.



Anecdote

One-lane Bridge

A former operating systems student of HMD's was one of the original designers of the IBM personal computer. When we studied deadlock in class, he shared a deadlock anecdote with me. Two

drivers were approaching a one-lane bridge. They reached the middle of the bridge at the same time, forcing a confrontation. They got out of their cars and went nose to nose. One driver

said, "I don't back up for idiots." The other driver calmly got back in his car and started backing up. He said, "No problem—I do."

Lesson to operating system designers: Processes are neither stubborn nor cooperative, so deadlock is a painful experience at best. Usually, one or more processes will have to be "backed out" of the deadlock, losing some or all of their work.

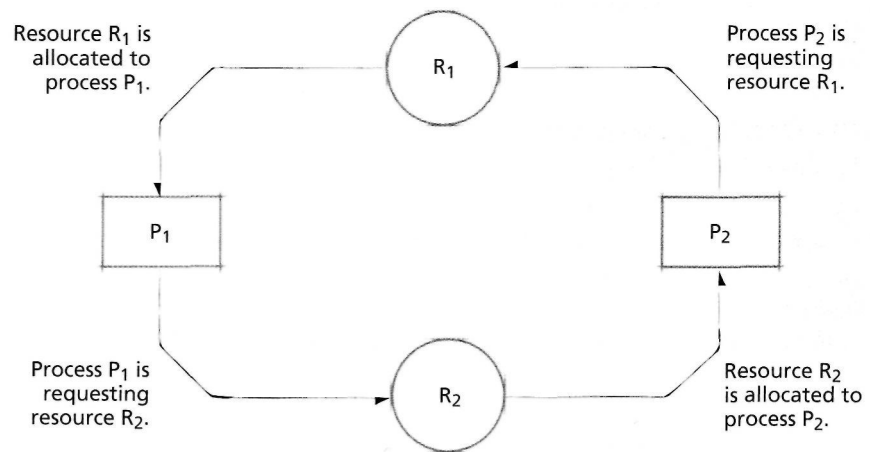


Figure 7.2 | Resource deadlock example. This System is deadlocked because each process holds a resource being requested by the other process and neither process is willing to release the resource it holds.

Self Review

1. If the traffic deadlock in Fig. 7.1 were to be represented using a resource-allocation graph what would the processes be? What would the resources be?
2. Suppose several arrows are pointing from a process to resources in a resource-allocation graph. What does that mean? How does it affect the possibility of deadlock?

Ans: 1) The cars would be the processes. The resources would be the sections of the street that the cars occupy. Each car currently holds the section of the street directly below it and is requesting the section in front of it. 2) It means that the process is requesting several resources. The possibility of deadlock depends on whether those resources are allocated to other processes, some of which, in turn, are requesting resources held by the process we are discussing.

7.2.3 Deadlock in Spooling Systems

A spooling system improves system throughput by disassociating a program from slow devices such as printers. For example, if a program sending lines to the printer must wait for each page to be printed before it can transmit the next page, then it will execute slowly. To speed the program's execution, a spooling system routes output pages to a much faster device, such as a hard disk, where they are temporarily stored until they can be printed.

Spooling systems can be prone to deadlock. Some spooling systems require that the complete output from a program be available before printing can begin. Several partially completed jobs generating pages to a spool file can become deadlocked if the disk's available space fills before any job completes. The user or system administrator may kill one or more jobs to make sufficient spooling space available for the remaining jobs to complete.

Typically, the system administrator specifies the amount of space for spooling files. One way to make deadlocks less likely is to provide considerably more space for spooling files than is likely to be needed. This solution may not be feasible if space is at a premium. A more common solution is to restrain the input spoolers so that they do not accept additional print jobs when the spooling files begin to reach some **saturation threshold**, such as 75 percent full. This may reduce system throughput, but it is the price paid to reduce the likelihood of deadlock.

Today's systems are more sophisticated than this. They might allow printing to begin before the job is completed so that a full, or nearly full, spooling file can begin emptying while a job is still executing. This concept has been applied to streaming audio and video clips, where the audio and video begin to play before the clips download fully. In many systems spooling space allocation has been made more dynamic, so that if existing space starts to fill, then more space may be made available.



Operating Systems Thinking

Waiting, Deadlock and Indefinite Postponement

Operating systems must manage many different types of waiting scenarios, which rely on an abundant use of queues to do so. Processes and threads must wait for a processor to become available before they can execute; they often wait for I/O requests to complete and for requested resources to become available. I/O requests themselves must wait for the I/O devices to become available. Processes and threads sharing data may have to wait until a process or thread currently accessing that data finishes and leaves a critical section. In the producer/consumer relationship, the producer passes information to the consumer via a shared region of memory; when that region

becomes full, the producer cannot continue to produce more information, so it must wait for the consumer to empty some data; when that region becomes empty the consumer must wait for the producer to generate more data.

You will see extensive examples of waiting scenarios throughout this book. Operating systems must manage waiting carefully to avoid two serious problems, namely deadlock and indefinite postponement, each of which is discussed in detail in this chapter. In a deadlock scenario, processes and threads are waiting for events that will never happen; a simple two-process deadlock has each process waiting on a resource that only one process at

a time can use and that the other process is holding. Both processes cease progressing; this can be fatal in mission-critical systems where human lives may be at stake. Indefinite postponement (also called starvation) typically occurs when a process is waiting in line and other processes, perhaps those of a higher priority, are allowed to get in line in front of the waiting process. A steady stream of arriving high-priority processes is said to indefinitely postpone the lower-priority process. This could be as dangerous as deadlock in a mission-critical system. So take waiting seriously—if it is mismanaged, system failures could occur.

Self Review

1. Suppose a spooling system has a saturation threshold of 75 percent and limits the maximum size of each file to 25 percent of the total spooling file size. Could deadlock occur in this system?
2. Suppose a spooling system has a saturation threshold of 75 percent and limits the maximum size of each file to 25 percent of the total spooling file size. Describe a simple way to ensure that deadlock will never occur in the system. Explain how this could lead to inefficient resource allocation.

Ans: 1) Yes, deadlock can still occur in this system. For instance, several jobs can begin transferring their outputs. When the spooling file reaches the 75 percent threshold, new jobs are not allowed. However, jobs that have begun are allowed to continue spooling, which may result in deadlock if there is insufficient space in the spooling file. **2)** A simple adjustment would be to allow only one job to continue spooling data when the file reaches the threshold. This would be inefficient because it would limit the maximum job size to much less than the available spooling space.

7.2.4 Example: Dining Philosophers

Dijkstra's problem of the **Dining Philosophers**,^{5, 6} illustrates many of the subtle problems inherent in concurrent programming. The problem is this:

Five philosophers sit around a circular table. Each leads a simple life alternating between thinking and eating spaghetti. In front of each philosopher is a dish of spaghetti that is constantly replenished by a dedicated wait staff. There are exactly five forks on the table, one between each adjacent pair of philosophers. Eating spaghetti (in the most proper manner) requires that a philosopher use both adjacent forks (simultaneously). Develop a concurrent program free of deadlock and indefinite postponement that models the activities of the philosophers.

If the solution to this problem were not free of deadlock and indefinite postponement, one or more philosophers would starve. The program must, of course, enforce mutual exclusion—two philosophers cannot use the same fork at once. A typical philosopher behaves as shown in Fig. 7.3.

The problems of mutual exclusion, deadlock and indefinite postponement lie in the implementation of method `eat`. Consider the simple, yet dangerous implementation in Fig. 7.4. Because the philosophers operate asynchronously and con-

```

1  void  typicalPhilosopher()
2  {
3      while ( true )
4      {
5          think();
6          eat();
7      } // end while
8
9  } // end typicalPhilosopher

```

Figure 7.3 | Dining philosopher behavior.

```

1 void eat()
2 {
3     pickUpLeftFork();
4     pickUpRightFork();
5     eatForSomeTime();
6     putDownRightFork();
7     putDownLeftFork();
8 } // eat

```

Figure 7.4 | Implementation of method eat

currently, it is possible for each philosopher to execute line 3 before any philosopher executes line 4. In this case, each philosopher will hold exactly one fork, and no forks will remain available on the table. The philosophers will all deadlock and starve.

One way to break the deadlock when each philosopher holds a left fork is to force one or more philosophers to put their left fork down so that another philosopher may grab it as a right fork. To implement this rule, `pickUpRightFork()` can specify that a philosopher puts down the left fork if that philosopher cannot obtain the right fork. However, in this case, it is possible (albeit unlikely) that each philosopher will pick up and put down its left fork repeatedly in tandem without ever obtaining the two forks it needs to eat spaghetti. In this case, the philosophers are not deadlocked, but "livelocked," suffering from indefinite postponement, and they will still starve. The solution must prevent deadlock (by forcing philosophers to put down forks) but also prevent livelock (indefinite postponement) by guaranteeing that each philosopher will obtain both forks from time to time. In the exercises, you will be asked to develop a complete solution to the Dining Philosophers problem.

Self Review

1. Could the implementation of the `eat` method in Fig. 7.4 allow the philosophers to live in harmony without starving?
2. Consider the `eat` method in Fig. 7.4. Suppose lines 3 and 4 were replaced with `pickUpBothForksAtOnce()`; would that implementation prevent deadlock? Would it prevent the philosophers from starving?

Ans: 1) Yes. We cannot predict the relative speeds of asynchronous concurrent processes. It is possible that not all five philosophers will pick up their left fork at once. Various philosophers could acquire the two forks they need, then release them to give the other philosophers a chance to eat. 2) Yes, it would prevent deadlock, assuming that `pickUpBothForksAtOnce()` can be performed atomically, but it would still allow indefinite postponement. This could occur, for example, if two of the philosophers were to continually pick up two forks, eat, put down the forks and pick them up again before any of the other philosophers could grab them; those other philosophers would starve.

7.3 *Related Problem: Indefinite Postponement*

In any system that requires processes to wait as a result of resource-allocation and process scheduling decisions, a process may be delayed indefinitely while other processes receive the system's attention. This situation, called indefinite postponement, indefinite blocking, or starvation, can be as devastating as deadlock.

Indefinite postponement may occur because of biases in a system's resource-scheduling policies. When resources are scheduled on a priority basis, it is possible for a given process to wait for a resource indefinitely, as processes with higher priorities continue to arrive. Waiting is a fact of life and is certainly an important aspect of what goes on inside computer systems, so systems should be designed to manage waiting processes fairly and efficiently. Some systems prevent indefinite postponement by increasing a process's priority gradually as it waits for a resource—this technique is called **aging**. Eventually, the waiting process's priority will exceed the priorities of all processes, and the it will be serviced.

Self Review

1. Dekker's Algorithm and Peterson's Algorithm prevented a process from being indefinitely postponed from entering its critical section. Describe how indefinite postponement was prevented. How is this related to aging?
2. Suppose an interactive process appears to be "dead in the water." Does this mean that the process is definitely deadlocked? Could it be indefinitely postponed? Are there other possibilities?

Ans: 1) Both Dekker's Algorithm and Peterson's Algorithm made the waiting process the favored process when the resource was available again. This is similar to aging, because the waiting process earns a higher priority by waiting. 2) This is an interesting question. From the user's perspective, an interactive process would appear "dead in the water" if the system simply stopped responding to the interactive user's requests. The process could be deadlocked, but it does not have to be. It could be suffering from indefinite postponement, but it does not have to be. It could also be waiting for an event that may happen shortly, it could be locked in an infinite loop or it could be doing a lengthy calculation that will soon finish. Anyone who has experienced a system that appears to "hang" knows how frustrating this can be. What services should the operating system provide to help the user in such situations? We address these issues throughout the book.

7.4 *Resource Concepts*

As a resource manager, the operating system is responsible for the allocation of a vast array of resources of various types; this is part of what makes operating system design so interesting. We consider resources that are **preemptible**, such as processors and main memory. Processors are perhaps the most frequently preempted resources on a computer system. Processors must be rapidly switched (i.e., multiplexed) among all active processes competing for system service to ensure that these processes progress at a reasonable rates. Whenever a particular process reaches a point at which it cannot effectively use a processor (such as during a long

wait for input/output completion), the operating system dispatches another process to that processor. As we will see in Chapters 9, 10 and 11, a user program currently occupying a particular range of locations in main memory may be removed or preempted by another program. Preemption is thus extremely critical to the success of multiprogrammed computer systems.

Certain resources are **nonpreemptible**; they cannot be removed from the processes to which they are assigned until the processes voluntarily release them. For example, tape drives and optical scanners are normally assigned to a particular process for periods of minutes or hours.

Some resources may be **shared** among several processes, while others are dedicated to a single process at a time. Although a single processor can normally belong to only one process at a time, its multiplexing among many processes creates the illusion of simultaneous sharing. Disk drives are sometimes dedicated to single processes, but typically they contain files that can be accessed by many processes. Disks can be multiplexed among many processes requesting I/O.

Data and programs certainly are resources that the operating system must control and allocate. On multiprogramming systems, many users may simultaneously want to use an editor program. If the operating system maintained in main memory a separate copy of the editor for each program, there would be a significant amount of redundant data, wasting memory. A better technique is for the operating system to load one copy of the code in memory and to make the copy available to each user. If a process were allowed to modify this shared code, other processes might behave unpredictably. As a result, this code must be **reentrant**, meaning the code is not modified as it executes. Code that may be changed but is reinitialized each time it is used is said to be **serially reusable**. Reentrant code may be shared by several processes simultaneously, whereas serially reusable code may be used correctly by only one process at a time.

When we call particular resources shared, we must be careful to state whether they may be used by several processes simultaneously by only one process at a time. The latter kind—serially reusable resources—are the ones that tend to become involved in deadlocks.

Self Review

1. (T/F) Processes do not deadlock as a result of contending for the processor.
2. (T/F) Nonpreemptible resources must be hardware.

Ans: 1) True. The processor is a preemptible resource that can easily be taken from a process, assigned to other processes and returned to the original process, enabling it to proceed normally. 2) False. Certain software resources are nonpreemptible, e.g., monitors.

7.5 Four Necessary Conditions for Deadlock

Coffman, Elphick and Shoshani⁷ proved that the following four conditions are necessary for deadlock to exist:

1. A resource may be acquired exclusively by only one process at a time (**mutual exclusion condition**).
2. A process that has acquired an exclusive resource may hold that resource while the process waits to obtain other resources (**wait-for condition**, also called the **hold-and-wait condition**).
3. Once a process has obtained a resource, the system cannot remove it from the process's control until the process has finished using the resource (**no-preemption condition**).
4. Two or more processes are locked in a "circular chain" in which each process is waiting for one or more resources that the next process in the chain is holding (**circular-wait condition**).

Because these are *necessary* conditions, the existence of a deadlock implies that each of them must be in effect. As we will see later, this observation helps us develop schemes to prevent deadlocks. Taken together, all four conditions are necessary and **sufficient** for deadlock to exist (i.e., if they are all in place, the system is deadlocked).

Self Review

1. Describe how the four necessary conditions for deadlock apply to spooling systems.
2. Which of the four conditions would be violated if a user could remove jobs from a spooling system?

Ans: 1) No two jobs can simultaneously write data to the same location in the spooling file. Partially spooled jobs remain in the spooling file until more space is available. Jobs cannot remove other jobs from the spooling file. Finally, when the spooling file is full, each job waits for all of the other jobs to free up space. 2) This would violate the "no-preemption" condition.

7.6 Deadlock Solutions

Deadlock has been one of the more productive research areas in computer science and operating systems. There are four major areas of interest in deadlock research—deadlock prevention, deadlock avoidance, deadlock detection and deadlock recovery.

In **deadlock prevention** our concern is to condition a system to remove any possibility of deadlocks occurring. Prevention is a clean solution as far as deadlock itself is concerned, but prevention methods can often result in poor resource utilization.

In **deadlock avoidance** the goal is to impose less stringent conditions than in deadlock prevention in an attempt to get better resource utilization. Avoidance methods do not precondition the system to remove all possibility of deadlock. Instead, they allow the possibility to loom, but whenever a deadlock is approached, it is carefully sidestepped.

Deadlock detection methods are used in systems in which deadlocks can occur. The goal is to determine if a deadlock has occurred, and to identify the processes and resources that are involved.

Deadlock recovery methods are used to clear deadlocks from a system so that it may operate free them, and so that the deadlocked processes may complete their execution and free their resources. Recovery is a messy problem at best, typically requiring that one or more of the deadlocked processes be flushed from the system. The flushed processes are normally restarted from the beginning when sufficient resources are available, with much or all of the work done by these processes being lost.

Self Review

1. Compare and contrast deadlock prevention and deadlock avoidance.
2. Some systems ignore the problem of deadlock. Discuss the costs and benefits of this approach.

Ans: 1.) Deadlock prevention makes deadlock impossible but results in lower resource utilization. With deadlock avoidance, when the threat of deadlock approaches, it is sidestepped and resource utilization is higher. Systems using either deadlock prevention or deadlock avoidance will be free of deadlocks. 2) Systems that ignore deadlock may fail when deadlock occurs. This is an unacceptable risk in mission-critical systems, but it may be appropriate in other systems where deadlocks rarely occur and the "cost" of dealing with an occasional deadlock is lower than the costs of implementing deadlock prevention or avoidance schemes.

7.7 Deadlock Prevention

This section considers various methods of deadlock prevention and examines the effects on both users and systems, especially from the standpoint of performance.^{8-9-10-11, 12, 13} Havender,¹⁴ observing that a deadlock cannot occur if a system denies any of the four necessary conditions, suggested the following deadlock prevention strategies:

- Each process must request all its required resources at once and cannot proceed until all have been granted.
- If a process holding certain resources is denied a further request, it must release its original resources and, if necessary, request them again together with the additional resources.
- A linear ordering of resources must be imposed on all processes; i.e., if a process has been allocated certain resources, it may subsequently request only those resources later in the ordering.

In the sections that follow, we consider each strategy independently and discuss how each denies one of the necessary conditions (see the Anecdote, No Nuts, Bolts or Screws Allowed). Note that Havender presents three strategies, not four. The first necessary condition, namely that processes claim exclusive use of the resources they require, is not one that we want to break, because we specifically want to allow **dedicated** (i.e., serially reusable) resources.

Self Review

1. What is the basic premise of Havender's research on deadlock prevention?

Ans: Deadlock cannot occur in systems in which any of the necessary conditions for deadlock are precluded.

7.7.1 Denying the "Wait-For" Condition

Havender's first strategy requires that all of the resources a process needs to complete its task must be requested at once. The system must grant them on an "all or none" basis. If all the resources needed by a process are available, then the system may grant them all to the process at once, and the process may continue to execute. If they are not all available, then the process must wait until they are. While the process waits, however, it may not hold any resources. Thus the "wait-for" condition is denied, and deadlocks cannot occur.

Although this strategy successfully prevents deadlock, it wastes resources. For example, a program requiring four tape drives at one point in its execution must request, and receive, all four before it begins executing. If all four drives are needed throughout the execution of the program, then there is no serious waste. But, suppose the program needs only one tape drive to begin execution (or worse yet, none



Anecdote

No Nuts, Bolts or Screws Allowed

Some years ago, HMD had the privilege of visiting one of the automated factories of one of the largest computer vendors. At the time, the factory—run almost exclusively by robots—was producing small inexpensive printers for personal computers. The head of the factory made a strong point. He observed that printers have a significant number of moving parts and that the company's engineers had traced failures of

previous lines of printers to nuts, bolts and screws that had loosened as a consequence of the mechanical motion. So when they designed the printer that was being produced at this factory, the design team made a pact that all the parts of this printer (there were about 80 of them including the casing) would be designed to snap into place—no nuts, bolts or screws would be allowed in the design. The head of the factory

then proceeded to assemble one of the printers by hand from the pieces sitting on his table. At the end of the demo, he plugged the printer into a wall socket and printed several pages to show that it was up and running. He mentioned that the "mean-time-to-failure" of this type of printer was proven to be miniscule compared to previous lines.

Lesson to operating systems designers: One way to minimize the chance of system failure is to eliminate as many "points of failure" as possible at the design stage. This works well with another design heuristic—KIS (keep it simple). Both of these heuristics are appealing, but can be difficult to implement. As you read this book, keep this anecdote in mind, searching for points of failure that you may be able to eliminate in future operating systems designs.

at all) and then does not need the remaining tape drives for several hours. The requirement that the program must request, and receive, all four tape drives before execution begins means that substantial resources will sit idle for several hours.

One approach to getting better resource utilization in these circumstances is to divide a program into several threads that run relatively independently of one another. Then resource allocation can be controlled by each thread rather than for the entire process. This can reduce waste, but it involves a greater overhead in application design and execution.

This Havender strategy could cause indefinite postponement if, for example, it favored waiting processes with small resource needs over those attempting to accumulate many resources. A steady stream of processes arriving with small resource needs could indefinitely postpone a process with more substantial needs. One way to avoid this is to handle the needs of the waiting processes in first-come-first-served order. Unfortunately, accumulating the full complement of resources for a process with substantial needs would cause considerable waste, as the gradually accumulating resources would sit idle until all were available.

In large mainframe computer environments with expensive resources, there is some controversy over whom to charge for these unused resources. Because the resources are being accumulated for a specific user, some designers feel this user should pay for them, even while they sit idle. Other designers say this would destroy the **predictability of resource charges**; if the user tried to run the process on a busy day, the charges would be much higher than when the machine was lightly loaded.

Self Review

1. Explain how denying the "wait-for" condition can reduce the degree of multiprogramming in a system.
2. Describe under what conditions a process can be indefinitely postponed when using Havender's scheme to deny the "wait-for" condition.

Ans: 1) If processes in a system require more resources than are available to concurrently execute, the system must force some of those processes to wait until others finish and return their resources. In the worst case, the system may be limited to executing only one process at a time. 2) A process could request a substantial number of a system's resources. If the operating system gives higher priority to processes that require fewer of system resources, then the process requesting all of the system's resources will be postponed indefinitely while processes that require fewer resources proceed.

7.7.2 Denying the "No-Preemption" Condition

Havender's second strategy denies the "no preemption" condition. Suppose a system does allow processes to hold resources while requesting additional resources. As long as sufficient resources remain available to satisfy all requests, the system cannot deadlock. But consider what happens when a request for additional resources cannot be satisfied. Now a process holds resources that a second process